FH Wedel · Master Informatik

Modul „Methoden des Machine Learning“ · Sommersemester 2026

Projektbericht

Webscraper für Modulhandbücher mit AWS

Skalierbare Erhebung und Klassifikation von
Modulhandbüchern deutscher Hochschulen

Jan Soetebeer · Sascha Lauk

Oktober 2026

Entstanden im Kontext der Forschungsgruppe KIWIT (BMFTR-gefördert), Forschungsprofil A (Makroebene)

Quellcode und Commit-Historie: `https://github.com/JanSoetebeer/KML`

Zusammenfassung

Dieser Bericht dokumentiert Entwicklung, Betrieb und Evaluation eines modularen Web-Crawlers, der öffentlich zugängliche Modulhandbücher deutscher Hochschulen erhebt und mit Methoden des Machine Learning von anderen PDF-Dokumenten unterscheidet. Der Bericht beschreibt nicht nur das Ergebnis, sondern vor allem den Weg dorthin: drei Architekturgenerationen auf AWS (von einer einzelnen Lambda-Funktion über Fargate-Bulk-Läufe bis zu einer Pipeline mit Discovery, OCR und LLM-Zweitstufe), die eingeschlagenen und wieder verworfenen Ansätze sowie die Rückschläge im Betrieb, darunter der Verlust der AWS-Sandbox und die Budget-Sperre.

Im Endstand wurden 164.396 verschiedene PDFs von 404 Hochschulen heruntergeladen (193.438 S3-Objekte, 154,1 GB; zwei Großläufe von 6,3 h und 12,0 h). Der TF-IDF-Klassifikator erreichte in der gruppierten Kreuzvalidierung eine Precision von 96,5 %, im realen Crawl jedoch nur 25 % (Score 0,5–0,9) bzw. 82,5 % (Score $\geq$ 0,9). Erst eine LLM-Zweitstufe (Claude Haiku 4.5 auf Amazon Bedrock) für 34.141 Dokumente brachte belastbare Qualität: 11.771 Modulhandbücher sind LLM-bestätigt, zusammen mit den ungeprüften Treffern oberhalb von Score 0,9 ergibt sich eine Endliste von 19.049 Dokumenten, von denen schätzungsweise 18.628 echte Modulhandbücher sind. 263 von 404 Hochschulen (65 %) sind abgedeckt. Der Bericht diskutiert Grenzen (Recall, LLM als Referenz, Kosten) und gibt eine Hochrechnung für eine deutschlandweite Vollerhebung.

Schlüsselwörter: Web Scraping, Scrapy, AWS Lambda, AWS Fargate, Dokumentklassifikation, TF-IDF, LLM-Review, Amazon Bedrock, Modulhandbücher.

Inhaltsverzeichnis

Zusammenfassung i

1 Einleitung 1
1.1 Forschungskontext: KIWIT . . . 1
1.2 Ausgangssituation und Motivation . . . 1
1.3 Problemstellung und Zielsetzung . . . 2
1.4 Leitfragen und Kennzahlen . . . 2
1.5 Vorgehen und Aufbau des Berichts . . . 3

2 Projektverlauf 4
2.1 Phasen im Überblick . . . 4
2.2 Wochenweise Aktivität . . . 5
2.3 Werkzeuge und Arbeitsweise . . . 8

3 Grundlagen und Technologiestack 9
3.1 Crawler: Scrapy . . . 9
3.2 Webapp: FastAPI und SQLite . . . 9
3.3 Klassifikation: TF-IDF statt Transformer . . . 10
3.3.1 Evaluationsmetriken . . . 10
3.4 Cloud-Komponenten . . . 10
3.5 Externe Dienste . . . 10

4 Architektur und ihre Entwicklung 12
4.1 Leitprinzipien . . . 12
4.2 Architektur v1: Lambda plus Webapp (Juni bis Anfang Juli) . . . 12
4.3 Architektur v2: Klassifikation, Feedback, CI/CD und Fargate (Juli) . . . 14
4.4 Architektur v3: Discovery, OCR und LLM-Zweitstufe (August bis Oktober) . . . 14
4.5 Vergleich der Versionen . . . 16
4.6 Datenfluss pro Dokument . . . 16
4.7 Modularität und Wiederverwendbarkeit . . . 16

5 Die AWS-Umgebung: Aufbau, Entwicklung und Rückschläge 19
5.1 Rahmenbedingungen: eine Sandbox auf Zeit . . . 19
5.2 Von Lambda zu Fargate (und warum nicht EC2 oder ECS) . . . 19
5.2.1 Ausgangsentscheidung . . . 19
5.2.2 Wo Lambda scheiterte . . . 19
5.2.3 Optionen . . . 19
5.2.4 Umsetzung des Fargate-Pfads . . . 20
5.2.5 Gerechnet versus gemessen . . . 20
5.3 Infrastruktur als Code: Terraform, CLI, Bootstrap . . . 20
5.4 CI/CD mit GitHub Actions . . . 21
5.5 Betrieb der Webapp . . . 21
5.6 Zugriffsbeschränkungen und ihre Folgen . . . 22
5.7 Rückschläge und Verluste . . . 22
5.8 Kosten und Kostenkontrolle . . . 24
5.9 Lehren aus dem AWS-Betrieb . . . 25

6 Der Crawler: Entwicklung und Recall-Arbeit **26**
6.1 Ausgangspunkt: ungesteuerter Tiefen-Crawl . . . 26
6.2 Fokussierter Crawl (24. Juli) . . . 26
6.2.1 Rauchtest auf vier Hochschulen . . . 27
6.3 Großlauf über die Hochschulliste (12. August) . . . 27
6.4 Schnelle Gewinne ohne Zusatzbudget . . . 28
6.5 Discovery: Seeds aus Suchindizes . . . 28
6.6 Seeds-only-Modus: Lücken ohne Duplikate schließen . . . 28
6.6.1 robots.txt und Seeds . . . 30
6.7 Phase 2: Vollständigkeit je Hochschule . . . 30
6.7.1 Deep Run (16./17. September) . . . 30
6.8 Höflichkeit und Grenzen . . . 30
6.9 Zusammenfassung der Crawl-Entwicklung . . . 31

7 Klassifikation: vom TF-IDF-Modell zur LLM-Zweitstufe **32**
7.1 Aufgabe und Daten . . . 32
7.2 Modell und Evaluation . . . 32
7.3 Der Praxis-Gap: 96,5 % im Training, 25 % im Crawl . . . 33
7.4 Human-in-the-Loop . . . 35
7.5 LLM-Zweitstufe . . . 35
7.6 OCR für gescannte PDFs . . . 37
7.7 Erwogene Alternativen . . . 37

8 Ergebnisse und Kennzahlen **39**
8.1 Kennzahlen auf einen Blick . . . 39
8.2 Wie lange dauert ein Scraping-Lauf? . . . 40
8.3 Wie viele Dokumente werden gefunden, und was geschieht mit ihnen? . . . 40
8.4 Wie groß sind die gespeicherten Datenmengen? . . . 40
8.5 Wie viele Hochschulen und Studiengänge werden erfasst? . . . 42
8.6 Wie unterscheiden sich kleine und große, öffentliche und private Hochschulen? . . . 43
8.7 Wie hoch ist die Erkennungs- und Klassifikationsquote? . . . 43
8.8 Wie viele Dokumente müssen manuell nachbearbeitet werden? . . . 46
8.9 Wie viele Duplikate gibt es? . . . 47
8.10 Wie weit reichen die ältesten Dokumente zurück? . . . 47
8.11 In welcher Tiefe der Webseiten liegen die Dokumente? . . . 48
8.12 Welche Datenmenge wäre bei einer deutschlandweiten Vollerhebung zu erwarten? 48

9 Diskussion: Sackgassen, Lehren und Grenzen **50**
9.1 Wege, die nicht (oder nur teilweise) funktioniert haben . . . 50
9.2 Lehren . . . 50
9.3 Grenzen und Bedrohungen der Validität . . . 52

10 Fazit und Ausblick **53**
10.1 Fazit . . . 53
10.2 Ausblick . . . 53
10.2.1 Qualität und Recall . . . 53
10.2.2 Inhaltliche Erschließung . . . 54
10.2.3 Plattform und Betrieb . . . 54
10.2.4 Anschluss an KIWIT . . . 54

A Konfiguration der Läufe **55**

B Metriken im Detail **56**

C Reproduzierbarkeit **57**

D Weitere Auswertungsgrafiken **58**

E Glossar **60**

Literaturverzeichnis **61**

Abbildungsverzeichnis

2.1 Zeitleiste des Projekts mit Phasen und Meilensteinen (Kalenderwochen 24–40, 2026). Die Phasen sind in den folgenden Abschnitten beschrieben. 4
2.2 Arbeitsaufwand von Jan Soetebeer je Kalenderwoche, eingefärbt nach Projektphase. 6

4.1 Architektur v1 (Juni bis 3. Juli): Webapp auf EC2 ruft Lambda synchron auf; S3 und DynamoDB halten den Zustand. 13
4.2 Architektur v2 (11.–24. Juli): Klassifikation im Container-Image, Feedback-Schleife, CI/CD und Fargate-Bulk-Pfad mit Routing nach URL-Anzahl. 15
4.3 Architektur v3 (August bis Oktober): Discovery-Seeds, Seeds-only-Modus, Fargate-Crawl mit Wachhund, OCR und LLM-Review auf S3-Daten sowie Bootstrap-Skript. 17
4.4 Datenfluss pro Dokument mit den Mengen der Abschlussauswertung (Endliste = LLM-bestätigt + TF-IDF $\geq$ 0,9 ungeprüft). 18

5.1 Cloud-Kosten nach Posten (Abrechnung, Juli bis 5. Oktober 2026). 24
5.2 Tageskosten und kumulierte Kosten. Die Spitzen entsprechen den Großläufen (12.08., 16.09.) und den Tagen der LLM-Prüfung (02.09., 30.09.–02.10.); die Grundlast ist die dauerhaft laufende Webapp. Die gestrichelte Linie markiert die Budgetgrenze von 50 $. 25

6.1 Abdeckung (links) und gelistete Positive (rechts) im Projektverlauf. Die grauen Balken sind TF-IDF-Positive ohne LLM-Prüfung; erst die Endstände (grün, blau) sind qualitätsgesichert. 31

7.1 Anteil der vom LLM bestätigten TF-IDF-Positive je Score-Bereich (LLM als Referenz). Die Bereiche oberhalb 0,9 stammen aus der Pilotstichprobe. 33
7.2 Precision im Training (gruppierte Kreuzvalidierung) gegenüber dem realen Crawl. Die Differenz ist die Kernaussage zur Übertragbarkeit. 34
7.3 Fehlerarten der TF-IDF-Positive, die das LLM verworfen hat. 34
7.4 Bestätigungsrate des LLM mit und ohne Handbuch-Stichwort im Dateinamen. 35
7.5 Precision (LLM-Bestätigungsrate) bei Score $\geq$ 0,9 auf verdächtigen und übrigen Hosts. 36
7.6 LLM-Läufe: Umfang und Bestätigungsrate. 37
7.7 OCR-Trichter: von den Scans ohne Textebene bis zu den bestätigten Handbüchern. 38

8.1 Trichter vom Crawl (164.396 PDFs) über die TF-IDF-Positive und die LLM-Prüfung bis zu den geschätzt echten Handbüchern. 39
8.2 Laufzeit, Dokumentenzahl und Durchsatz der drei Läufe. 40
8.3 Durchsatz (neue PDFs pro Stunde) im Verlauf der beiden Großläufe. 40
8.4 Endstatus aller Dokumente nach Lauf. 41
8.5 Vergleich von August-Lauf und Deep Run (URL-Überlappung). 41
8.6 Gespeicherte Daten je Lauf und Dateigrößenverteilung der PDFs (logarithmische Achse). 42
8.7 Verteilung der Treffer pro Hochschule. 42
8.8 Die 20 ergiebigsten Hosts. 43
8.9 Abdeckung nach Hochschultyp. 44

8.10 Abdeckung nach Trägerschaft. 44
8.11 Abdeckung nach Hochschulgröße (Studierendenzahl). 44
8.12 Mittlere Treffer, mittlere PDFs und Anteil ergiebiger Hochschulen nach Größenklasse. 44
8.13 Studierendenzahl gegen Treffer pro Hochschule. 45
8.14 Abdeckung nach Bundesland. 45
8.15 Was vorliegt und was fehlen könnte (grobe Schätzung). 46
8.16 Manueller Prüfaufwand ohne und mit LLM-Zweitstufe (30 s je Dokument). . . . 47
8.17 Duplikate in der Endliste. 47
8.18 Jahreszahlen in den Dateinamen der gelisteten Dokumente. 48
8.19 Pfadtiefe der Fundseiten (Näherung für die Klicktiefe) für alle PDFs und die gelisteten Modulhandbücher. 48
8.20 Lineare Hochrechnung der Erhebung für 40.000 Modulhandbücher (Ist: durchgezogen, Hochrechnung: schraffiert). 49

D.1 Score-Verteilung nach Endstatus (logarithmische Achse). 58
D.2 Konfidenzverteilung der LLM-Urteile. 58
D.3 Status der Hochschulen vor und nach der Bereinigung. 59

Tabellenverzeichnis

2.1 Wochenübersicht Jan Soetebeer. 5
2.2 Wochenübersicht Sascha Lauk. 7

3.1 Technologiestack im Überblick (Stand Projektende). 9

4.1 Die drei Architekturversionen im Vergleich. 16

5.1 Erwogene Rechenpfade für den Crawl und ihr Schicksal. 20
5.2 Rechnung im MVP-Vortrag gegenüber den realen Großläufen. 20
5.3 Betriebsvorfälle und Gegenmaßnahmen. 23

6.1 Rauchtest auf vier Hochschulen (24. Juli). 27
6.2 Erprobte Discovery-Quellen. 29
6.3 Crawl-Strategien im Zeitverlauf mit Problem, Maßnahme und Wirkung. 31

7.1 Modellversionen und gruppierte Kreuzvalidierung. 33
7.2 LLM-Läufe der Zweitstufe. 36

8.1 Zentrale Kennzahlen des Projekts. 39

9.1 Verworfene oder gescheiterte Wege und ihre Lehre. 51

10.1 Akzeptanzkriterien und Ergebnis. 53

A.1 Konfigurationsgrenzen des Crawlers: Lambda/MVP gegenüber Fargate Deep Run. 55

B.1 Metriken der Modellvarianten (gruppierte Kreuzvalidierung), jeweils am Recall-Ziel von 95 %. 56

1 Einleitung

1.1 Forschungskontext: KIWIT

Das vorliegende Projekt ist im Kontext der Forschungsgruppe KIWIT (*Funktionen und Folgen Künstlicher Intelligenz in der Wissenschafts- und Hochschulorganisation*) entstanden, die seit April 2025 vom Bundesministerium für Forschung, Technologie und Raumfahrt (BMFTR) gefördert wird. Die Beschreibung des Kontexts stammt vom Betreuer des Projekts:

> „Das vorliegende Projekt ist im Kontext der Forschungsgruppe KIWIT (Funktionen und Folgen Künstlicher Intelligenz in der Wissenschafts- und Hochschulorganisation) entstanden, die seit April 2025 vom Bundesministerium für Forschung, Technologie und Raumfahrt (BMFTR) gefördert wird. KIWIT untersucht empirisch, wie Künstliche Intelligenz Strukturen und Prozesse in Wissenschaft und Hochschulorganisationen verändert — auf der Ebene der Forschungspraxis, der institutionellen Steuerung sowie des Lehrens und Lernens.
>
> Innerhalb dieses Rahmens ist der entwickelte Crawler dem Forschungsprofil A (Makroebene) zugeordnet und dient der systematischen Erhebung öffentlich zugänglicher Hochschulinformationen. Universitätswebseiten sind eine zentrale primäre Datenquelle, um zu erfassen, wie Hochschulen KI-bezogene Strategien, Richtlinien und Angebote kommunizieren und formalisieren. Das modular aufgebaute Tool ermöglicht eine skalierbare, reproduzierbare Datenerhebung über eine Vielzahl von Institutionen hinweg und bildet damit eine methodische Grundlage für die quantitativen Analysevorhaben innerhalb von KIWIT.“

Daraus ergibt sich eine Anforderung, die den gesamten Projektverlauf geprägt hat: Das System soll *keine Einmallösung* für genau eine Dokumentart sein, sondern eine *wiederverwendbare Erhebungsplattform*. Modulhandbücher dienen als erster, gut überprüfbarer Anwendungsfall. Sie eignen sich dafür, weil sie auf den Webseiten fast aller Hochschulen als PDF vorliegen, aber nirgends zentral verzeichnet sind, und weil sich ihre Echtheit vergleichsweise eindeutig beurteilen lässt.

1.2 Ausgangssituation und Motivation

An der FH Wedel werden regelmäßig neue Studienangebote konzipiert. Für eine fundierte Entscheidungsgrundlage ist der Vergleich mit Studiengängen anderer Hochschulen unerlässlich, da deren Modulhandbücher verlässlich Auskunft über Lehrinhalte, Umfänge und Prüfungsformen geben. Bislang mussten solche Dokumente manuell auf den Webseiten der Hochschulen gesucht und gesichtet werden. Das ist zeitaufwändig und fehleranfällig: In einer Aufwandsabschätzung zum MVP wurden 10 bis 15 Minuten je Hochschule veranschlagt, für einen Durchlauf über alle Hochschulen also 40 bis 60 Stunden, und das Ergebnis müsste in jedem Semester neu erhoben werden.

Die Problemstellung ist dabei weniger das Herunterladen von Dokumenten, das Scrapy seit Jahren beherrscht, als das *Finden* und *Aussortieren*: Hochschulwebseiten sind sehr unterschiedlich aufgebaut, Handbücher liegen auf Fakultäts-Subdomains, in Modul-Datenbanken oder hinter skriptgenerierten Download-Links, und Prüfungsordnungen, Flyer oder Vorlesungsverzeichnisse sehen Modulhandbüchern täuschend ähnlich.

1.3 Problemstellung und Zielsetzung

Als User Story formuliert: *Als Mitarbeiter der FH Wedel möchte ich einen Webscraper, der übergebene Links automatisiert nach Modulhandbüchern durchsucht, relevante von irrelevanten Dokumenten unterscheidet und die gefundenen Modulhandbücher strukturiert speichert, damit ich Lehrinhalte und -umfänge unterschiedlicher Hochschulen im Vergleich zu unseren Studienangeboten analysieren kann.*

Daraus wurden zu Projektbeginn vier Akzeptanzkriterien abgeleitet:

A1 Der Scraper durchsucht alle übergebenen Links vollständig nach Dokumenten, die als Modulhandbuch identifizierbar sind.

A2 Alle tatsächlichen Modulhandbücher werden gefunden — keine relevanten Dokumente werden übersehen (Recall).

A3 Dokumente, die keine Modulhandbücher sind, werden zuverlässig erkannt und nicht gespeichert (Precision).

A4 Die identifizierten Modulhandbücher werden strukturiert und für die Weiterverarbeitung nutzbar abgelegt.

Der Projektumfang hat sich im Verlauf deutlich verschoben. Der MVP im Juli arbeitete mit rund 6.400 PDFs von etwa 239 Hochschulen und einem Klassifikator, der auf 202 Dokumenten von 21 Hochschulen trainiert war. Ab August lautete das Ziel, *möglichst alle* Modulhandbücher *aller* 404 Hochschulen der Hochschulliste zu erheben. Als grobe Fachschätzung für die Größenordnung diente ein Erwartungswert von etwa 40.000 Modulhandbüchern. Dieser Wert ist keine Messung, sondern nur Referenz für die Recall-Einordnung.

1.4 Leitfragen und Kennzahlen

Neben der Funktion soll der Bericht den Daten- und Forschungsaspekt des Projekts greifbar machen. Dazu werden folgende Fragen mit Zahlen beantwortet (Kapitel 8):

- Wie lange dauert ein Scraping-Lauf, und wie viele Dokumente und Daten fallen dabei an?
- Wie viele Hochschulen werden erfasst, und wie unterscheiden sich kleine und große, öffentliche und private Hochschulen?
- Wie hoch ist die Erkennungs- bzw. Klassifikationsquote, und wie viele Dokumente müssen manuell nachbearbeitet werden?
- Wie viele Duplikate treten auf, wie alt sind die ältesten Dokumente, und in welcher Tiefe der Webseiten liegen die meisten Funde?
- Welche Datenmenge, Laufzeit und welche Kosten wären bei einer deutschlandweiten Vollerhebung zu erwarten?

1.5 Vorgehen und Aufbau des Berichts

Der Bericht folgt dem tatsächlichen Projektverlauf und stellt *den Weg* gleichberechtigt neben das Ergebnis. Kapitel 2 gibt den zeitlichen Überblick mit wochenweiser Aufschlüsselung der Aktivitäten. Kapitel 3 fasst die eingesetzten Technologien zusammen. Kapitel 4 zeigt die Architektur in drei Versionen, Kapitel 5 die Entwicklung der AWS-Umgebung einschließlich der Probleme (Sandbox-Verlust, Budget-Sperre). Kapitel 6 und 7 beschreiben Crawler und Klassifikation mit allen eingeschlagenen Wegen. Kapitel 8 stellt die Kennzahlen dar, Kapitel 9 diskutiert Sackgassen, Lehren und Grenzen. Kapitel 10 schließt mit Fazit und Ausblick.

Quellcode. Das gesamte Projekt (Crawler, Klassifikator, Webapp, Infrastruktur, Auswertung und dieser Bericht) liegt in einem öffentlichen Git-Repository: `https://github.com/JanSoetebeer/KML`. Die Commit-Historie belegt den zeitlichen Verlauf, der in Kapitel 2 beschrieben ist.

Lesehinweis zu Farben in Diagrammen. In den Architekturdiagrammen sind *neu hinzugekommene* Komponenten blau hinterlegt, verworfene Ansätze orange gestrichelt. In den Ergebnisgrafiken gilt ein festes Schema: blau = LLM-bestätigt, grün = TF-IDF $\geq 0{,}9$ (nicht LLM-geprüft), orange = verworfen, grau = negativ.

2 Projektverlauf

Das Projekt lief von der ersten Zeile Code am 10. Juni 2026 bis zur Abschlussauswertung am 2. Oktober 2026, also über 17 Kalenderwochen. Rückblickend lässt es sich in vier Phasen gliedern, die sich jeweils durch ein dominierendes Problem auszeichnen (Abb. 2.1).

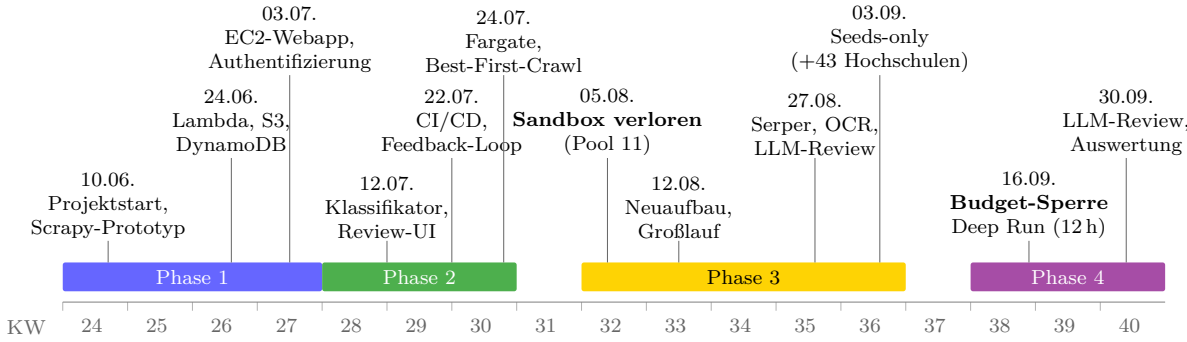


Abbildung 2.1: Zeitleiste des Projekts mit Phasen und Meilensteinen (Kalenderwochen 24–40, 2026). Die Phasen sind in den folgenden Abschnitten beschrieben.

2.1 Phasen im Überblick

Phase 1: Prototyp und AWS-Grundlage (KW 24–27, Juni bis Anfang Juli). Ein Scrapy-Spider, der PDF- und Word-Dokumente von einer Start-URL herunterlädt, wird um einen `JobRunner` (ein Job je URL, parallel begrenzt) und einen persistenten Visited-Store erweitert. Das erste Architekturdiagramm entsteht (Architektur v1, Abschnitt 4.2). Danach folgt die Cloud-Anbindung: Lambda-Container-Image in ECR, S3 für Dokumente, DynamoDB für den Visited-Store, anschließend Webapp mit Authentifizierung auf einer EC2-Instanz. Dominierendes Thema: *überhaupt etwas Lauffähiges in die Cloud bringen.*

Phase 2: Klassifikator und Plattform (KW 28–30, Juli). Der Endlosschleifen-Bug bei Mehrfach-Formaten wird behoben, Jobs laufen asynchron mit Fortschrittsanzeige, ein Terraform-Stack soll die Infrastruktur beschreiben. Der erste TF-IDF-Klassifikator wird trainiert und in die Scraping-Pipeline integriert. Es folgen die Human-in-the-Loop-Feedbackschleife, CI/CD über GitHub Actions und, als Antwort auf das 15-Minuten-Limit von Lambda, der Fargate-Bulk-Pfad (Architektur v2, Abschnitt 4.3). Dominierendes Thema: *Qualität und Skalierung des Crawls.*

Phase 3: Großlauf und Recall-Analyse (KW 32–36, August bis Anfang September). Am 5. August wird der Verlust der AWS-Sandbox bemerkt: Daten, Konfiguration und der erste Großlauf über 411 Hochschulen sind weg. Nach Zugang zu einem neuen Pool wird die Umgebung am 12. August über ein Bootstrap-Skript neu aufgebaut, und noch am selben Tag wird der Großlauf in der neuen Umgebung wiederholt. Die Auswertung zeigt, dass nur ein Teil der erwarteten Handbücher gefunden wurde. Es entstehen das Evaluationsmodul, die Discovery (Common Crawl, Serper, Firecrawl), der OCR-Fallback, die LLM-Zweitstufe und der Seeds-only-Modus (Architektur v3, Abschnitt 4.4). Dominierendes Thema: *Recall.*

Phase 4: Deep Run und LLM-Bereinigung (KW 38–40, September bis Oktober). Nach der Budget-Sperre der Sandbox wird der Crawl um Breitensuche und Laufzeit-Wachhund erweitert und als zwölfstündiger Deep Run gestartet (164.396 PDFs kumuliert). Weil die TF-IDF-Positive in großer Zahl falsch sind, werden rund 34.141 Dokumente von einem LLM geprüft, Scans per OCR nachbearbeitet und die Abschlussauswertung mit 23 Grafiken erstellt. Dominierendes Thema: *Precision*.

2.2 Wochenweise Aktivität

Tabelle 2.1 schlüsselt die Aktivität von Jan Soetebeer wochenweise auf. Die Stunden und Tätigkeiten lassen sich anhand der Commit-Historie im Repository nachvollziehen (28 Commits auf `main`, 42 inklusive Nebenzweigen). Den Entwurf des Frontends (Aufbau der Oberfläche) hat Sascha Lauk erstellt; die Umsetzung in Code lag bei Jan Soetebeer.

Tabelle 2.1: Wochenübersicht Jan Soetebeer.

Woche	Zeitraum	Tätigkeiten	Std.
KW 24	08.06.–14.06.	Projektstart; Scrapy-Prototyp (DocumentSpider, PDF/Word-Download); JobRunner mit parallelen Jobs; Visited-Store; erstes Architekturdiagramm (Architektur v1)	10
KW 25	15.06.–21.06.	Einarbeitung in die AWS-Umgebung (Lambda, ECR, IAM), aufwändig und komplex	5
KW 26	22.06.–28.06.	AWS-Integration: Lambda-Container-Image, ECR, S3-Pipeline, DynamoDB-Visited-Store, IAM-Policies; Frontend-Grundgerüst (Login, Tabs) umgesetzt nach dem Entwurf von Sascha	12
KW 27	29.06.–05.07.	Authentifizierung (bcrypt, Session-Token); Hosting der Webapp auf EC2 mit Docker Compose und Caddy; Rollen- und Modellverwaltung	12
KW 28	06.07.–12.07.	Endlosschleifen-Bug (Timeout bei Mehrfach-Formaten) behoben; asynchrone Jobs mit Fortschrittsanzeige; Terraform-Stack; Tiefen-Crawl; Ergebnis- und Review-UI; erster Klassifikator (TF-IDF + logistische Regression, Gruppen-Kreuzvalidierung); Scraper-Integration	20
KW 29	13.07.–19.07.	MVP-Präsentation erstellt (15.07.-16.07.); Deployment-Versuche Lambda-Image und EC2; Klassifikator in Lambda-Image eingebettet	10
KW 30	20.07.–26.07.	Human-in-the-Loop-Feedbackschleife (Review-UI, S3-Feedback, Nachtraining); CI/CD mit GitHub Actions (OIDC, SSM); CSV-Import der Hochschulliste; Skalierungs-Roadmap; fokussierter Crawl (Best-First, Sitemap, Subdomains); Extraktionsprofile; Fargate-Bulk-Pfad (Architektur v2)	22
KW 31	27.07.–02.08.	Kleine Fixes und Bugbehebung	2
KW 32	03.08.–09.08.	Verlust der Sandbox (Pool 11) am 05.08. bemerkt: Daten, Konfiguration und der erste Großlauf weg; Planung der Evaluation	3
KW 33	10.08.–16.08.	Neuaufbau der AWS-Umgebung in neuem Pool (Bootstrap-Skript); Wiederholung des Großlaufs über 411 Hochschulen (6,3 h); Evaluationsmodul mit HTML-Bericht	15

Woche	Zeitraum	Tätigkeiten	Std.
KW 34	17.08.–23.08.	Recall-Analyse (145 Hochschulen ohne Treffer); Schwellenwert-Anpassung; Discovery-Modul (Common Crawl); OCR-Fallback	8
KW 35	24.08.–30.08.	Serper- und Firecrawl-Anbindung; LLM-Zweitstufe auf Amazon Bedrock; End-to-End-Test auf Beispielhochschulen	9
KW 36	31.08.–06.09.	Webapp-Schalter für Discovery/Rendering; parallele Bedrock-Aufrufe; Nachtraining mit 5.000+ Modulhandbüchern; Seeds-only-Modus; Per-Hochschule-Diagnose; Seeds-only-Lauf (+43 Hochschulen)	12
KW 37	07.09.–13.09.	keine dokumentierte Aktivität (abwesend)	0
KW 38	14.09.–20.09.	Budget-Sperre der Sandbox analysiert; Robots-Bypass für Discovery-Seeds; Fehler in der Evaluation (LLM-Urteile überschrieben) behoben; Breadth-Fairness und Watchdog; Deep Run (12 h, 132.193 PDFs)	14
KW 39	21.09.–27.09.	Urlaub	0
KW 40	28.09.–04.10.	LLM-Review von rund 34.100 Dokumenten; OCR von 3.553 Scans; Pilotstichprobe und Host-Analyse; Abschlussauswertung mit 24 Grafiken; Abschlusspräsentation und Projektbericht	16
Summe			**170**

Abbildung 2.2 zeigt dieselben Werte als Säulendiagramm. Die Spitzen liegen in KW 30 (CI/CD, Feedback-Loop, fokussierter Crawl und Fargate an zwei Tagen), KW 28 (Klassifikator und Review-UI) sowie in KW 33 (Sandbox-Neuaufbau und Wiederholung des Großlaufs). In KW 37 und KW 39 war Jan Soetebeer abwesend bzw. im Urlaub; das ist für die Budget-Sperre in Kapitel 5 relevant. Insgesamt sind es 170 Stunden (Phase 1: 39 h, Phase 2: 54 h, Phase 3: 47 h, Phase 4: 30 h).

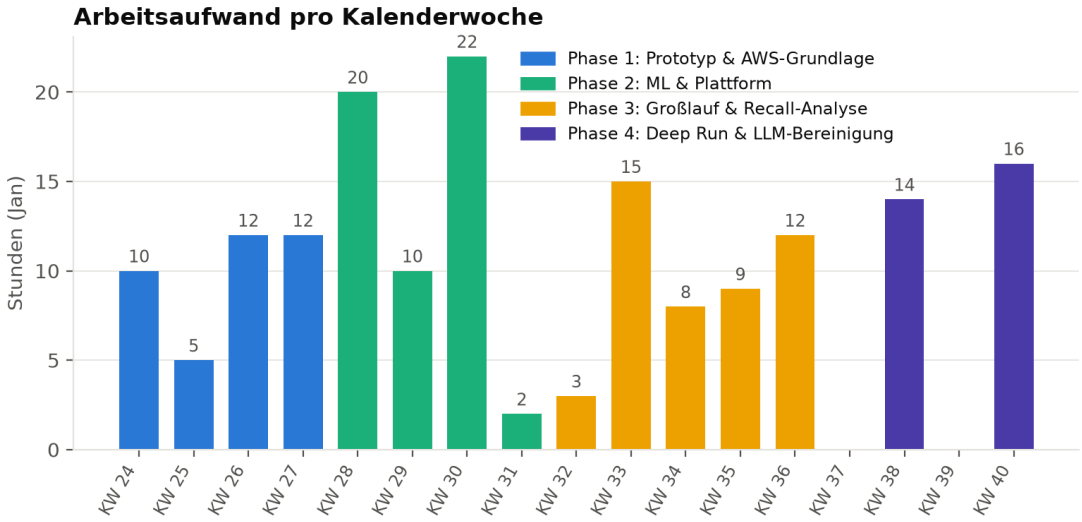


Abbildung 2.2: Arbeitsaufwand von Jan Soetebeer je Kalenderwoche, eingefärbt nach Projektphase.

Wochenübersicht Sascha Lauk

Tabelle 2.2 zeigt die Aktivität von Sascha Lauk; insgesamt sind es 166 Stunden.

Tabelle 2.2: Wochenübersicht Sascha Lauk.

Woche	Zeitraum	Tätigkeiten	Std.
KW 24	08.06.–14.06.	Projektstart; Scrapy-Prototyp (DocumentSpider, PDF/Word-Download); JobRunner mit parallelen Jobs; Visited-Store; erstes Architekturdiagramm (Architektur v1)	20
KW 25	15.06.–21.06.	Einarbeitung in die AWS-Umgebung (Lambda, ECR, IAM), aufwändig und komplex	4
KW 26	22.06.–28.06.	Erstellung des Frontend Designs; Frontend-Grundgerüst (Login, Tabs) umgesetzt nach dem Entwurf von Sascha	25
KW 27	29.06.–05.07.	Authentifizierung (bcrypt, Session-Token); Hosting der Webapp auf EC2 mit Docker Compose und Caddy; Rollen- und Modellverwaltung	10
KW 28	06.07.–12.07.	Endlosschleifen-Bug (Timeout bei Mehrfach-Formaten) behoben; asynchrone Jobs mit Fortschrittsanzeige; Terraform-Stack; Tiefen-Crawl; Ergebnis- und Review-UI; erster Klassifikator (TF-IDF + logistische Regression, Gruppen-Kreuzvalidierung); Scraper-Integration	8
KW 29	13.07.–19.07.	MVP-Präsentation erstellt (15.07.-16.07.); Deployment-Versuche Lambda-Image und EC2; Klassifikator in Lambda-Image eingebettet	12
KW 30	20.07.–26.07.	Human-in-the-Loop-Feedbackschleife (Review-UI, S3-Feedback, Nachtraining); CI/CD mit GitHub Actions (OIDC, SSM); CSV-Import der Hochschulliste; Skalierungs-Roadmap; fokussierter Crawl (Best-First, Sitemap, Subdomains); Extraktionsprofile; Fargate-Bulk-Pfad (Architektur v2)	13
KW 31	27.07.–02.08.	Kleine Fixes und Bugbehebung	3
KW 32	03.08.–09.08.	Verlust der Sandbox (Pool 11) am 05.08. bemerkt: Daten, Konfiguration und der erste Großlauf weg; Planung der Evaluation	4
KW 33	10.08.–16.08.	Neuaufbau der AWS-Umgebung in neuem Pool (Bootstrap-Skript); Wiederholung des Großlaufs über 411 Hochschulen (6,3 h); Evaluationsmodul mit HTML-Bericht	10
KW 34	17.08.–23.08.	Recall-Analyse (145 Hochschulen ohne Treffer); Schwellenwert-Anpassung; Discovery-Modul (Common Crawl); OCR-Fallback	7
KW 35	24.08.–30.08.	Serper- und Firecrawl-Anbindung; LLM-Zweitstufe auf Amazon Bedrock; End-to-End-Test auf Beispielhochschulen	7
KW 36	31.08.–06.09.	Webapp-Schalter für Discovery/Rendering; parallele Bedrock-Aufrufe; Nachtraining mit 5.000+ Modulhandbüchern; Seeds-only-Modus; Per-Hochschule-Diagnose; Seeds-only-Lauf (+43 Hochschulen)	10
KW 37	07.09.–13.09.	Händische Bewertung Falschklassifizierten Dokumenten	8
KW 38	14.09.–20.09.	Budget-Sperre der Sandbox analysiert; Robots-Bypass für Discovery-Seeds; Fehler in der Evaluation (LLM-Urteile überschrieben) behoben; Breadth-Fairness und Watchdog; Deep Run (12 h, 132.193 PDFs)	10
KW 39	21.09.–27.09.	Prüfung des Codes auf Verbesserungsmöglichkeiten	2
KW 40	28.09.–04.10.	Abschlusspräsentation und Projektbericht	13
Summe			**166**

2.3 Werkzeuge und Arbeitsweise

Entwickelt wurde in Python 3.12 mit Git/GitHub (Branches `main`, `develop`, `feature/add-ML-Models`, Pull-Requests), Docker und der AWS-CLI. Für die Coding-Aufgaben wurde ein KI-Assistent (Claude Code) eingesetzt. In der LLM-Zweitstufe der Pipeline selbst kommt Claude Haiku 4.5 über Amazon Bedrock zum Einsatz (Abschnitt 7.5).

3 Grundlagen und Technologiestack

Leitend für alle Technologieentscheidungen war das Anforderungsprofil eines projektbasierten, aber auf Wiederverwendbarkeit ausgelegten Systems: geringe Betriebskosten, wenig Infrastrukturaufwand, kurze Entwicklungszeit und eine Architektur, die sich sauber erweitern lässt. Tabelle 3.1 gibt den Gesamtüberblick; die Begründungen der wichtigsten Entscheidungen folgen in den Abschnitten.

Tabelle 3.1: Technologiestack im Überblick (Stand Projektende).

Bereich	**Technologie und Rolle**	**Seit**
Crawler	Scrapy (Spider, Pipelines, Dupe-Filter, AutoThrottle); generische Engine mit austauschbaren Extraktionsprofilen	Juni
Webapp	FastAPI + uvicorn, statisches HTML/CSS/JavaScript, SQLite (Nutzer, Rollen, Modelle), bcrypt, Caddy als HTTPS-Proxy	Juni/Juli
Rechnen	AWS Lambda (interaktiv, $\leq$ 10 URLs) und AWS Fargate (Bulk); gleiches Container-Image	Juni/Juli
Zustand	S3 (Dokumente, Manifeste, Feedback, Seeds), DynamoDB (Visited-Store)	Juni
Klassifikation	scikit-learn: TF-IDF (Wort- und Zeichen-n-Gramme) + logistische Regression, PyMuPDF zur Textextraktion	Juli
Zweitstufe	Claude Haiku 4.5 über Amazon Bedrock (spezifikationsgesteuerter Prompt), Tesseract-OCR für Scans	August
Discovery	Common Crawl (CDX-Index), Serper (Google-SERP-API), Firecrawl (JS-Rendering)	August
Betrieb	GitHub Actions (OIDC, SSM), Bootstrap-Skript (PowerShell + AWS-CLI), Docker Compose; Terraform nur zeitweise	Juli/August
Auswertung	Python (matplotlib, stdlib-Evaluationspaket), HTML-Berichte, LaTeX	August–Oktober

3.1 Crawler: Scrapy

Scrapy liefert die Bausteine für robustes, wiederholtes Crawlen: einen Dupe-Filter innerhalb eines Laufs, konfigurierbare Drosselung (AutoThrottle) und eine Pipeline-Architektur, in die sich Verarbeitungsschritte als austauschbare Klassen einhängen lassen. Genau diese Erweiterbarkeit war ausschlaggebend: Ein neuer Spider, eine neue Pipeline-Stufe oder ein neues Visited-Store-Backend sind jeweils eine zusätzliche Klasse. Die Höflichkeitsregeln (robots.txt nach RFC 9309 [3], Verzögerung, identifizierbarer User-Agent) sind Standard, nicht Option.

3.2 Webapp: FastAPI und SQLite

FastAPI liefert REST-API und statisches Frontend aus demselben Prozess; es entfällt eine separate Frontend-Infrastruktur. Router trennen die Domänen (`auth`, `users`, `roles`, `models`, `scrape`). Als Datenhaltung genügt SQLite auf einem Docker-Volume, weil nur ein Prozess schreibt und die Nutzerzahl einstellig ist; das Schema wird idempotent beim Start angelegt. Passwörter liegen

als bcrypt-Hash vor, Sitzungen sind signierte 12-Stunden-Tokens. Das Rollenmodell trennt die *Position* (Administrator/User) von fachlichen *Rollen* (zunächst nur „Modulhandbuch“), und Modelle werden über dieselben Rollen Nutzern zugeordnet. Damit lässt sich ein neuer Anwendungsfall mit einer neuen Rolle und einem passenden Modell abbilden, ohne Frontend-Code zu ändern.

3.3 Klassifikation: TF-IDF statt Transformer

Für „Ist dieses PDF ein Modulhandbuch?“ wurde bewusst eine klassische Pipeline gewählt: TF-IDF [5] über Wort- und Zeichen-n-Gramme und logistische Regression [4]. Zu Projektbeginn standen 202 Dokumente aus 21 Hochschulen zur Verfügung; dafür ist das Finetuning eines Transformers die falsche Werkzeugklasse, während ein regularisiertes lineares Modell zuverlässig funktioniert, in Sekunden trainiert ist und ohne GPU in Lambda und Fargate läuft. Der eigentliche Flaschenhals ist Netzwerk-I/O und PDF-Textextraktion, nicht Matrixmultiplikation. Dass diese Entscheidung für die Massenfilterung richtig, für die Endqualität aber nicht ausreichend war, zeigt Kapitel 7.

3.3.1 Evaluationsmetriken

Bei ungleichgewichtigen Klassen ist die Fläche unter der Precision-Recall-Kurve (PR-AUC, hier als *average precision*) die Leitmetrik. Precision $P = \frac{TP}{TP+FP}$ gibt an, welcher Anteil der als Modulhandbuch gemeldeten Dokumente wirklich eines ist; Recall $R = \frac{TP}{TP+FN}$, welcher Anteil der echten gefunden wurde. Unsicherheiten von Anteilen werden als Wilson-Intervall [6] angegeben. Die Schwellen werden nicht auf 0,5 gesetzt, sondern aus den Daten auf ein Recall-Ziel von 95 % kalibriert, weil ein übersehenes Handbuch eine Datenlücke ist, ein Fehlalarm dagegen nur wenige Sekunden Review kostet. Die Entscheidung ist dreistufig: `automatic_negative`, `needs_review` und `automatic_positive`.

3.4 Cloud-Komponenten

Die Wahl von AWS ergab sich aus dem bereitgestellten Hochschul-Account (AWS-Sandbox der FH Wedel). Warum zunächst Lambda gewählt wurde und warum es später durch Fargate ergänzt werden musste, ist Gegenstand von Kapitel 5. Wichtig für die Plattformidee: *ein* Container-Image (Crawler + `mlclassifier` + Modell) läuft in allen Laufzeitumgebungen (Lambda, Fargate, lokal), nur der Einstiegspunkt unterscheidet sich.

3.5 Externe Dienste

- **Common Crawl** [2]: freier Webindex; über den CDX-Index lassen sich pro Domain PDF-URLs mit „modul/handbuch“-Mustern finden, ohne die Hochschul-Server zu belasten.
- **Serper**: Google-SERP-API für Abfragen der Form `site:<domain> <Begriff>`; erreicht tief verlinkte Dateien, die Common Crawl nicht kennt. Es konnte nur die kostenlose Stufe genutzt werden.
- **Firecrawl**: rendert JavaScript-Seiten und liefert Link-Karten; im Projekt vorbereitet, aber nur ansatzweise getestet.

- **Amazon Bedrock** mit Claude Haiku 4.5 [1]: Zweitstufen-Klassifikation für die Grauzone.
- **Tesseract**: OCR für gescannte PDFs ohne Textebene.

4 Architektur und ihre Entwicklung

Die Architektur ist nicht in einem Wurf entstanden, sondern in drei Generationen gewachsen, die jeweils auf ein konkretes Problem der vorherigen reagiert haben. Dieses Kapitel zeigt jede Version als Diagramm (neue Komponenten blau, verworfene orange) und begründet die Übergänge. Die zugehörigen AWS-Details (Dienste, IAM, Kosten, Betriebsprobleme) stehen in Kapitel 5.

4.1 Leitprinzipien

Über alle Versionen hinweg gelten vier Prinzipien:

1. **Steuerung und Last trennen.** Das „Cockpit“ (Webapp auf EC2) ist klein, läuft dauerhaft und skaliert nicht mit der Datenmenge. Die Arbeit läuft nur bei Bedarf in kurzlebigen Containern.
2. **Ein Image, mehrere Laufzeiten.** Derselbe Container-Code läuft in Lambda, Fargate und lokal; der Einstiegspunkt (Lambda-Handler bzw. `bulk_run.py`) unterscheidet sich.
3. **Generische Engine, austauschbare Profile.** Die Crawl-Engine kennt keine Modulhandbücher; sie delegiert vier Entscheidungen (Linkbewertung, Zieldefinition, Extraktion aus Binärdateien, Extraktion aus Seiten) an ein *Extraktionsprofil*. Ein neuer Anwendungsfall ist ein neues Profil plus ein Registry-Eintrag.
4. **Zustand extern halten.** Dokumente, Manifeste, Feedback, Seeds und Visited-Store liegen in S3 bzw. DynamoDB, nicht im Container.

4.2 Architektur v1: Lambda plus Webapp (Juni bis Anfang Juli)

Die erste Version ist ein klassischer Serverless-Ansatz (Abb. 4.1). Die Webapp auf einer EC2-Instanz (FastAPI, SQLite, Caddy für HTTPS) nimmt eine URL oder eine CSV-Liste entgegen und ruft pro Anfrage eine Lambda-Funktion synchron auf. Die Funktion startet den `JobRunner`, der pro URL einen Scrapy-Spider betreibt (Tiefe 2, höchstens 60 Seiten), die Dateien über die Pipelines nach S3 schreibt und die besuchte URL im DynamoDB-Visited-Store vermerkt.

Begründung. Die Last ist selten, stoßweise und perfekt parallelisierbar: 424 Hochschulen haben nichts miteinander zu tun. Lambda skaliert auf null und kostet ohne Last nichts. Rechnerisch ergaben sich für einen Vollauf 424 parallele Invocations (innerhalb des Limits von 1.000), ein Rechenaufwand von rund 76.000 GB-Sekunden und damit etwa 1,30 USD. Im MVP-Vortrag wurde ein Vollauf auf rund zehn Minuten veranschlagt; diese Zahl war *gerechnet, nicht gemessen* (zur späteren Realität siehe Abschnitt 5.2).

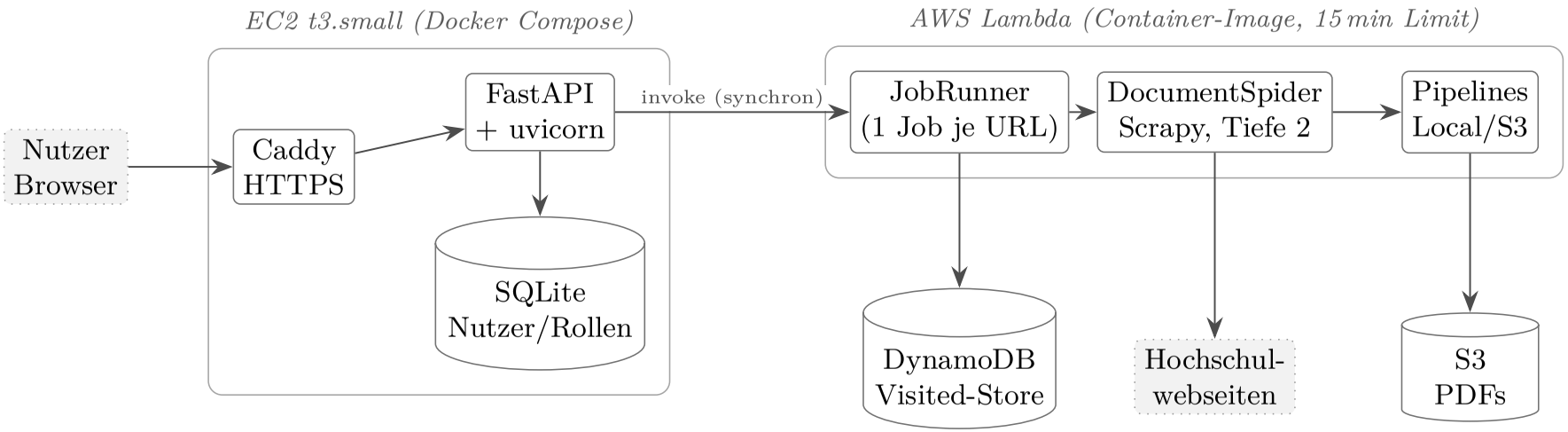


Abbildung 4.1: Architektur v1 (Juni bis 3. Juli): Webapp auf EC2 ruft Lambda synchron auf; S3 und DynamoDB halten den Zustand.

Grenzen. Drei harte Lambda-Eigenschaften erwiesen sich als Problem: das Zeitlimit von 15 Minuten (Tiefe/Seitenzahl mussten klein bleiben, sonst Timeout), das Puffern jeder Datei im Arbeitsspeicher und die Tatsache, dass alle URLs eines Aufrufs sich ein Budget teilen. Die synchrone Antwort der Webapp lief bei längeren Läufen zudem zuverlässig in den HTTP-Timeout; deshalb wurde das Polling-Muster (POST liefert eine `job_id`, GET fragt den Status) eingeführt.

4.3 Architektur v2: Klassifikation, Feedback, CI/CD und Fargate (Juli)

Die zweite Version (Abb. 4.2) fügt vier Dinge hinzu:

1. **Klassifikation im Crawl.** Eine `ClassificationPipeline` bewertet jedes heruntergeladene PDF im Speicher mit dem TF-IDF-Modell, annotiert das Item und schreibt ein Manifest (eine JSONL-Zeile je Dokument mit Score und Entscheidung) nach S3. Das Modell ist in das Container-Image eingebettet; Training und Betrieb nutzen dasselbe Paket (`mlclassifier`), ein Train/Serve-Skew ist damit ausgeschlossen.
2. **Human-in-the-Loop.** Die Review-UI zeigt Positive und Zweifelsfälle; Verdikte („Modulhandbuch“/„Kein Modulhandbuch“) werden gebündelt nach `s3://.../feedback/` geschrieben. Ein lokales Kommando (`feedback-retrain`) zieht die Verdikte, holt die Dokumente und trainiert nach.
3. **CI/CD.** Ein Push auf `main` baut das Image und aktualisiert Lambda (GitHub Actions, OIDC statt statischer Schlüssel) und rollt die Webapp über SSM Run Command aus, ohne dass ein SSH-Port geöffnet wird.
4. **Fargate-Bulk-Pfad.** Die Webapp routet nach URL-Anzahl: bis `BULK_URL_THRESHOLD` (zunächst 10) an Lambda, darüber an einen ECS-Fargate-Task mit demselben Image und dem Einstiegspunkt `bulk_run.py`. Der Task hat kein Zeitlimit, mehr Speicher und liest die URL-Liste aus S3.

Im selben Zeitraum wurde die Crawl-Engine modularisiert (Best-First-Frontier, Sitemap-Seeding, Subdomain-Erkennung, Extraktionsprofile; Kapitel 6) und das Terraform-Projekt angelegt, das später verworfen wurde (Abschnitt 5.3).

4.4 Architektur v3: Discovery, OCR und LLM-Zweitstufe (August bis Oktober)

Die Auswertung des ersten Großlaufs (Kapitel 6) zeigte, dass *Finden* das Hauptproblem ist, später, dass *Aussortieren* es ist. Die dritte Version (Abb. 4.3) antwortet auf beides:

- **Discovery-Modul.** Common Crawl und Serper liefern vorab URL-Kandidaten (*Seeds*) pro Hochschule; sie liegen in S3 und werden vom Fargate-Task als priorisierte Ziele geladen. Der *Seeds-only-Modus* holt ausschließlich diese Seeds und überspringt den Visited-Store, sodass nur Lücken-Hochschulen ohne Duplikate nachgearbeitet werden. Für Seeds lässt sich robots.txt gezielt umgehen (Abschnitt 6.6.1).
- **Crawl-Verbesserungen.** Breadth-Fairness (Seitenkappe je Fakultäts-Sektion), größere Tiefe und Seitenzahl sowie ein Laufzeit-Wachhund, der den Task hart beendet.
- **Nachbereitung auf dem Laptop.** OCR (Tesseract) für gescannte PDFs, LLM-Review über Amazon Bedrock und die Auswertungsskripte arbeiten direkt auf S3-Daten.

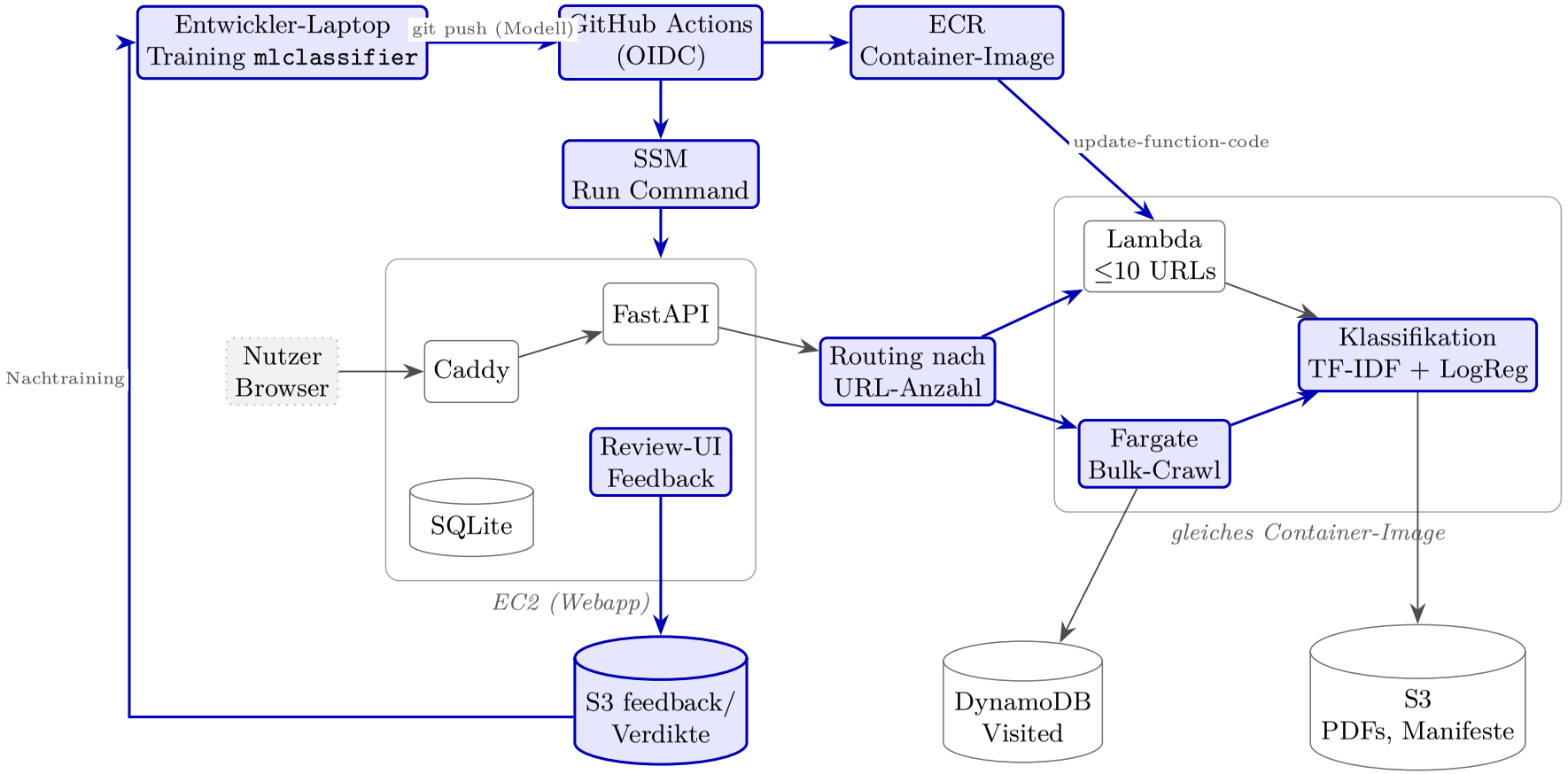


Abbildung 4.2: Architektur v2 (11.–24. Juli): Klassifikation im Container-Image, Feedback-Schleife, CI/CD und Fargate-Bulk-Pfad mit Routing nach URL-Anzahl.

- **Bootstrap-Skript.** Ein idempotentes PowerShell-Skript baut die gesamte AWS-Umgebung neu auf und ersetzt Terraform; es entstand als Reaktion auf den Sandbox-Verlust (Kapitel 5).
- **Verworfene Quellen** (orange): Google Custom Search und DuckDuckGo (Abschnitt 6.5).

4.5 Vergleich der Versionen

Tabelle 4.1 stellt die drei Versionen gegenüber. Auffällig ist, wie sich der Engpass verschoben hat: von der *Plattform* (läuft es in der Cloud?) über die *Laufzeit* (reicht Lambda?) und den *Recall* (wird alles gefunden?) hin zur *Precision* (ist es wirklich ein Modulhandbuch?).

Tabelle 4.1: Die drei Architekturversionen im Vergleich.

	v1 (Juni–Juli)	**v2 (Juli)**	**v3 (August–Oktober)**
Rechnen	Lambda (synchron)	Lambda + Fargate, Routing nach URL-Anzahl	Fargate-Bulk, Seeds-only; Nachbereitung lokal
Crawl	Tiefe 2, 60 Seiten, ungesteuert	Best-First, Sitemap, Subdomains, Profile	Seeds, Sektions-Cap, Tiefe 5, 2.500 Seiten
Klassifikation	–	TF-IDF im Image, Review-Band	+ LLM-Zweitstufe, OCR
Qualitätsschleife	–	Human-in-the-Loop, Nachtraining	LLM-Labels, Pilotstichprobe
Deployment	manuell, Terraform-Entwurf	GitHub Actions (OIDC, SSM)	+ Bootstrap-Skript für Neuaufbau
Hauptproblem danach	15 min, Speicher, wenig Funde	Recall (nur 261 von 404 Hochschulen)	Precision, Kosten, Betrieb

4.6 Datenfluss pro Dokument

Unabhängig von der Version durchläuft jedes Dokument dieselbe Kette (Abb. 4.4): Download, Textextraktion, Bewertung durch das TF-IDF-Modell und je nach Score entweder direkte Übernahme, LLM-Prüfung oder Verwurf. Scans ohne Textebene gehen zunächst in die OCR und anschließend in dieselbe LLM-Prüfung. Die Zahlen sind die der Abschlussauswertung.

4.7 Modularität und Wiederverwendbarkeit

Die Forderung des Projektkontexts (Kapitel 1), eine Plattform statt einer Einzellösung zu liefern, wurde an vier Stellen umgesetzt:

- **Extraktionsprofile** (`profiles/`): `modulhandbuch` (Schlüsselwort-gewichtet, Referenzprofil), `html_content` (Vorlage für HTML-Inhalte und strukturierte Felder). Der Engine-Code bleibt unverändert.
- **Spezifikationsgesteuerter LLM-Review** (`specs.py`): Eine `ClassificationSpec` (Definition, Beispiele, Ausschlüsse) beschreibt, was gesucht wird; das Urteilsfeld ist generisch (`is_match`).

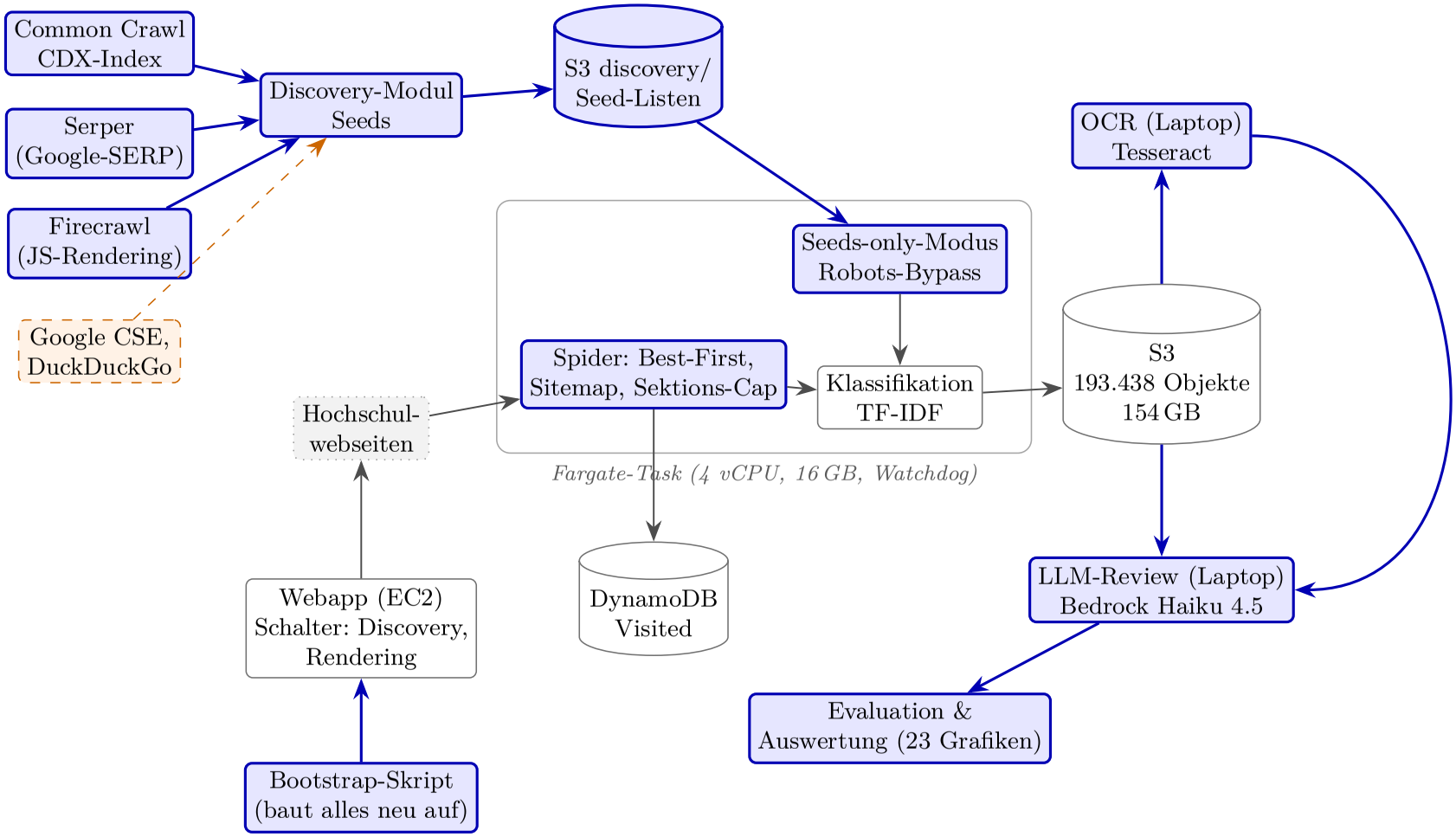


Abbildung 4.3: Architektur v3 (August bis Oktober): Discovery-Seeds, Seeds-only-Modus, Fargate-Crawl mit Wachhund, OCR und LLM-Review auf S3-Daten sowie Bootstrap-Skript.

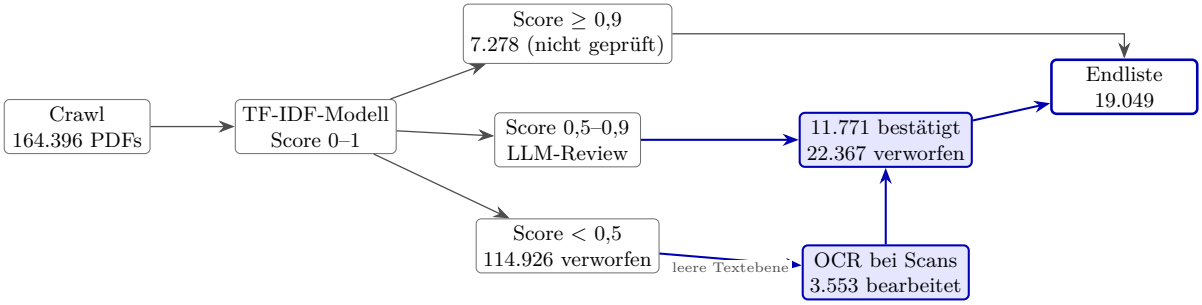


Abbildung 4.4: Datenfluss pro Dokument mit den Mengen der Abschlussauswertung (Endliste = LLM-bestätigt + TF-IDF $\geq$ 0,9 ungeprüft).

- **Profilgetriebene Discovery:** Suchbegriffe und URL-Stämme stammen aus dem Profil; ein neuer Dokumenttyp braucht nur ein anderes Profil.
- **Rollen-Modell-Matrix** der Webapp: neue Rolle plus neues Modell ergibt einen neuen Anwendungsfall ohne Frontend-Änderung.

Das ist die technische Brücke zu KIWIT: Sollen künftig statt Modulhandbüchern etwa KI-Strategiepapiere oder -Richtlinien erhoben werden, sind ein neues Profil, eine neue Spezifikation und ein neu trainiertes Modell nötig, nicht aber ein neuer Crawler.

5 Die AWS-Umgebung: Aufbau, Entwicklung und Rückschläge

Die AWS-Umgebung war nicht nur Hosting, sondern ein zentraler Teil der Projektarbeit. Dieses Kapitel beschreibt, wie sich Rechenumgebung und Betrieb entwickelt haben (von Lambda zu Fargate, von Terraform zu einem Bootstrap-Skript), welche Beschränkungen des Hochschul-Accounts den Weg bestimmt haben und welche Rückschläge aufgetreten sind.

5.1 Rahmenbedingungen: eine Sandbox auf Zeit

Alle Ressourcen liefen in einer AWS-Sandbox der FH Wedel (Region `eu-central-1`, Zugang per SSO-Rolle). Drei Eigenschaften dieser Umgebung haben das Projekt wesentlich geprägt:

- **Befristete Lease:** Läuft die Sandbox ab, wird *alles* gelöscht. Die Konto-ID ändert sich (im Projekt von `660941536751` zu `081757578883`).
- **Budgetdeckel:** Für den Sandbox-Pool galt ein Limit von 50 USD; bei Überschreitung wird der Pool eingefroren.
- **Service Control Policies (SCP):** Einzelne Dienste und Aktionen sind organisationsweit gesperrt (Abschnitt 5.6).

5.2 Von Lambda zu Fargate (und warum nicht EC2 oder ECS)

5.2.1 Ausgangsentscheidung

Die ursprüngliche Begründung für Lambda war schlüssig: Scrape-Jobs sind selten, stoßweise und perfekt parallel, Lambda skaliert auf null, ein Dauerbetrieb über ECS oder EC2 würde „Abschalten bei Nichtnutzung“ erst nachbauen müssen. Im MVP-Vortrag wurde ECS/Fargate ausdrücklich als Lösung für ein Problem bezeichnet, das man nicht habe. Das stimmte für den Ausgangszustand (Jobs auf zehn Minuten gedeckelt, einige Dutzend URLs), nicht mehr für das Zielbild „alle Hochschulen, tief gecrawlt“.

5.2.2 Wo Lambda scheiterte

Beim Versuch, Tiefe und Seitenzahl zu erhöhen (Tiefe 1–3, 25–200 Seiten wurden durchprobiert), liefen die Jobs ins 15-Minuten-Limit. Am 22. Juli entstand deshalb eine Skalierungs-Roadmap. Sie identifizierte drei harte Grenzen von Lambda: (1) das Zeitlimit von 15 Minuten, (2) die Pufferung jeder Datei im Arbeitsspeicher (`upload_fileobj(BytesIO(...))`) und (3) die gemeinsame Zeit- und Speicherbudgetierung aller URLs eines Aufrufs. Visited-Store (DynamoDB) und Ausgabe (S3) skalierten dagegen problemlos; das Nadelöhr war ausschließlich die Crawl-Rechenzeit.

5.2.3 Optionen

Tabelle 5.1 fasst die erwogenen Rechenpfade zusammen.

Tabelle 5.1: Erwogene Rechenpfade für den Crawl und ihr Schicksal.

Option	Bewertung	Ergebnis
Lambda (nur)	Billig, schnell, aber 15 min und RAM-Pufferung; ungeeignet für tiefe Crawls	behalten für $\leq$ 10 URLs (interaktiv)
SQS-Fan-out auf Lambda	Eine Hochschule je Invocation, hunderte parallel; löst „viele Hochschulen“, nicht „eine sehr tiefe Hochschule“	geplant (Roadmap, Phase 1b), *nicht gebaut*
EC2-Stopgap	Großer Lauf auf der vorhandenen Webapp-Instanz; keine neue Infrastruktur, aber t3.small und belegt Dauerbetrieb	nur als Übergangslösung beschrieben
ECS auf EC2	Cluster-Verwaltung nötig, Leerlaufkosten	nicht gewählt
ECS Fargate	Kein Zeitlimit, bis 4 vCPU/16 GB, gleiches Image, Abrechnung nur während der Laufzeit	**gewählt** (24. Juli)

5.2.4 Umsetzung des Fargate-Pfads

Am 24. Juli entstand der Bulk-Pfad innerhalb eines Tages: Einstiegspunkt `bulk_run.py` (umweltvariablengesteuert), Task-Definition und IAM-Rollen als Vorlagen mit Platzhaltern, GitHub-Workflow `bulk-crawl.yml` (`workflow_dispatch`) und ein URL-Routing in der Webapp. Die Defaults wurden schrittweise erhöht (2 vCPU/8 GB $\rightarrow$ 4 vCPU/16 GB; 300 $\rightarrow$ 800 Seiten; 15 $\rightarrow$ 30 Subdomain-Sitemaps). Hinzu kamen später ein Seeds-Modus und ein Wachhund (Abschnitt 5.8). Eine Stolperfalle: Die Webapp scheiterte zunächst mit „TaskDefinition not found“, weil nur der Workflow die Task-Definition selbst registrierte; das Bootstrap-Skript registriert sie nun immer.

5.2.5 Gerechnet versus gemessen

Tabelle 5.2 stellt die Annahmen des MVP-Vortrags den tatsächlichen Läufen gegenüber. Die Lambda-Rechnung (10 Minuten für alle Hochschulen) beschrieb einen *flachen* Crawl mit Tiefe 2 und 60 Seiten; das Ziel „möglichst vollständig“ erfordert dagegen Stunden. Die Kosten blieben trotzdem klein, weil Fargate nur während des Laufs abgerechnet wird.

Tabelle 5.2: Rechnung im MVP-Vortrag gegenüber den realen Großläufen.

	MVP-Rechnung (Juli)	Aug-Lauf	Deep Run
Laufzeit gesamt	$\sim$ 10 min (gerechnet)	6,3 h (gemessen)	12,0 h (gemessen)
Hochschulen	424	411	alle Hochschulen, Breitensuche
Seiten je Hochschule	60	800	2.500
Rechenkosten (Fargate, 4 vCPU/16 GB)	$\sim$ 1,30 $ (Lambda)	$\sim$ 3,70 $ (Abrechnung)	$\sim$ 3,30 $ (Abrechnung)
Heruntergeladene PDFs	–	65.944	132.193

5.3 Infrastruktur als Code: Terraform, CLI, Bootstrap

Terraform (11. Juli). Zunächst wurde ein Terraform-Stack angelegt (EC2, Elastic IP, Security Group, IAM, S3, DynamoDB, ECR, Lambda). In der Praxis scheiterte der Weg an der Umgebung:

Terraform und Docker waren in der Entwicklungsumgebung nicht ohne Weiteres nutzbar, der Terraform-Zustand und die handangelegten Ressourcen liefen auseinander, und die Wirkung des Stacks war schwer nachzuvollziehen. Der Stack wurde am 24. Juli als *deprecated* markiert; CI/CD und alle späteren Ressourcen (Fargate, IAM) liefen über AWS-CLI und eingecheckte JSON-Vorlagen mit Platzhaltern (`ACCOUNT_ID`, `REGION`).

Bootstrap-Skript (12. August). Die Sandbox-Löschung (Abschnitt 5.7) machte die Reproduzierbarkeit zum Hauptthema. Das Skript `webscraper/deploy/aws_bootstrap.ps1` baut die gesamte Umgebung idempotent aus einer leeren Konto-ID auf: S3, DynamoDB, ECR, Lambda-Rolle, Image-Build und -Push, Lambda-Funktion, EC2 (IAM, Security Group, Key Pair, Instanz, Elastic IP; die Instanz konfiguriert sich über User-Data), GitHub-OIDC-Rolle, Fargate (Cluster, Rollen, Egress-Security-Group, Task-Definition). Am Ende gibt es die beiden GitHub-Variablen aus, die gesetzt werden müssen. Einige Lektionen sind im Skript fest verankert:

- Die Richtlinien-Vorlagen müssen *reine* Platzhalter enthalten. Zwei Vorlagen enthielten noch die alte Konto-ID (ein Leak im öffentlichen Repository, der zwar keinen Zugriff gewährt, aber behoben wurde).
- Ersetzungen laufen case-sensitiv (`-creplace`), sonst wird der kleingeschriebene Schlüssel `awslogs-region` der Task-Definition beschädigt.
- Das Skript muss reines ASCII sein, weil Windows PowerShell 5.1 Dateien ohne BOM als CP1252 liest und ein Gedankenstrich die Tokenisierung zerstört.
- Neue IAM-Rollen sind nicht sofort verwendbar; die Lambda-Erstellung wiederholt bis zu sechsmal im Abstand von zehn Sekunden.

5.4 CI/CD mit GitHub Actions

Drei Workflows liegen im Repository: `deploy-lambda.yml` (Image bauen, nach ECR pushen, Lambda aktualisieren, Pfadfilter auf `webscraper/` und `mlclassifier/`), `deploy-webapp.yml` (Instanz per Tag finden, per SSM-Run-Command `remote_update.sh` ausführen: `git reset -hard`, `docker compose up -d -build`) und `bulk-crawl.yml` (Fargate-Task starten). Die Authentifizierung läuft per OIDC (kein statischer Schlüssel im Repository), die Anbindung per SSM (kein offener SSH-Port).

Beim ersten Einsatz am 22. Juli traten mehrere Fehler in kurzer Folge auf, jeweils mit einem eigenen Commit („fixed deploy“, „set home variable for git in ssm deploy“): Das Skript fand das App-Verzeichnis auf der Instanz nicht (Auto-Erkennung für drei mögliche Pfade nötig), und Git verlangte unter SSM eine gesetzte `HOME`-Variable. Für den interaktiven Zugang scheiterte `aws ssm start-session` zunächst, weil das Session-Manager-Plugin lokal fehlte.

5.5 Betrieb der Webapp

Die Webapp läuft als Docker-Compose-Stack (Webapp und Caddy) auf einer t3.small mit Elastic IP; die Datenbank liegt auf einem benannten Volume und überlebt Deployments. Daraus folgt, dass das Admin-Passwort das bei der Erstanlage gesetzte bleibt; ein nicht-destruktives Zurücksetzen per SQL-Update im Container wurde dokumentiert. Zwei Betriebsvorfälle sind erwähnenswert: Am 2. September war die Instanz per SSM nicht erreichbar („TargetNotConnected“, nach Ablauf der Sitzungsdaten), ein Reboot behob das. Außerdem erreichten neu

eingeführte Umgebungsvariablen (Discovery, Rendering) den Container zunächst nicht, weil sie in der `docker-compose.yml` nicht durchgereicht wurden.

5.6 Zugriffsbeschränkungen und ihre Folgen

- **Bedrock-Mantle gesperrt.** Eine SCP verbietet `bedrock-mantle` (403), erlaubt aber das native `bedrock:InvokeModel`. Die LLM-Anbindung wurde daher auf den nativen Client umgestellt, mit dem EU-Inference-Profil `eu.anthropic.claude-haiku-4-5-20251001-v1:0`.
- **Keine langlebigen Schlüssel.** Der Account erlaubt nur kurzlebige Bedrock-API-Schlüssel (rund 12 Stunden), die SSO-Sitzung verfällt teils alle 30 Minuten. Die LLM- und OCR-Läufe mussten deshalb in kleine, wiederaufnehmbare Blöcke („Chunks“) zerlegt werden; ein Skript (`next_chunk.py`) schneidet nur den noch offenen Rest und überspringt bereits geprüfte URLs.
- **Kostensicht gesperrt.** Cost Explorer (`ce:GetCostAndUsage`) ist per SCP verboten; die Kosten stammen aus dem exportierten Kostenbericht (Abschnitt 5.8).
- **TLS-Interception.** Im Firmennetz ersetzt ein Proxy Zertifikate; AWS-Aufrufe aus Python scheiterten mit `CERTIFICATE_VERIFY_FAILED`. Lokale Läufe nutzen `-no-verify-ssl`, die EC2-Instanz und Fargate dürfen die Prüfung nicht abschalten.
- **Bedrock-Kontingente.** Neue Konten melden zunächst 403 („account currently being verified“, nach unter zwei Stunden behoben). Am 30. September führte das Tageskontingent (`429 Too many tokens per day`) zum Abbruch der LLM-Läufe, die am Folgetag fortgesetzt wurden.

5.7 Rückschläge und Verluste

Tabelle 5.3 listet die schwerwiegenden Vorfälle. Die ersten beiden haben das Projekt am stärksten beeinflusst.

Der Sandbox-Verlust im Detail. Am 5. August um 11:30 Uhr fiel auf, dass der Sandbox-Pool 11 abgelaufen war; es kann auch schon früher passiert sein. Zu diesem Zeitpunkt wurde die Evaluation des ersten Großlaufs über die 411 Hochschulen vorbereitet. Alles in AWS war weg: Konfiguration ebenso wie Daten. Verloren gingen nicht nur Rechenressourcen, sondern auch *Daten*: der S3-Bucket mit allen Manifesten und den von Menschen vergebenen Verdikten der Feedback-Schleife sowie die Webapp-Datenbank. Dass Training und Code lokal beziehungsweise in Git lagen, begrenzte den Schaden. Die Konsequenz war doppelt: technisch die Idempotenz des Bootstrap-Skripts, organisatorisch die Regel, dass *nichts Wertvolles nur in der Sandbox* liegt. Alle späteren Manifeste und Zwischenergebnisse (`*.jsonl`) liegen deshalb zusätzlich lokal. Der Großlauf, auf den sich die Evaluation bezog, lag in der verlorenen Umgebung. Nach Zugang zu einem neuen Pool am 12. August wurde er noch am selben Tag (13:17–19:35 UTC) wiederholt; die gesamte weitere Auswertung beruht auf diesem zweiten Lauf.

Tabelle 5.3: Betriebsvorfälle und Gegenmaßnahmen.

Datum	Ereignis	Folge und Gegenmaßnahme
05.08.	**Sandbox-Lease (Pool 11) abgelaufen**, am 05.08. um 11:30 bemerkt (evtl. früher passiert); neuer Pool erst später	Alles in AWS weg: Daten, Konfiguration, S3 (Manifeste, Feedback-Verdikte, der erste Großlauf), DynamoDB, Lambda, ECR, EC2 samt Webapp-Datenbank. Erhalten blieben Git-Repository, lokale Trainingsdaten und das Modell. Hartkodierte Konto-IDs in den Richtlinien wurden durch Platzhalter ersetzt.
12.08.	Neuaufbau in Pool 12	Bootstrap-Skript baut die Umgebung neu auf; der Großlauf über die Hochschulliste wird noch am selben Tag wiederholt.
ab 05.09.	**Budgetsperre** (50 USD) des Sandbox-Pools, bemerkt am 16.09.	Das Budget war durch die erste LLM-Prüfung am 02.09. (21,61 $ an einem Tag, kumuliert 48 $) und die Dauerlast der Webapp faktisch am 05.09. erreicht. Eine Prüfung aller Regionen ergab keinen hängenden Task, keine verwaisten EBS-Volumes, IPs oder NAT-Gateways. Gegenmaßnahme: dreistufige Terminierungsgarantie (siehe unten).
22.07.	CI/CD scheitert beim ersten Deployment (SSM, `HOME`, App-Pfad)	vier Korrektur-Commits innerhalb von rund 20 Minuten.
03.09.	DynamoDB-Visited-Store überspringt bereits gecrawlte Hochschulen	Discovery-Wiederholung fand nichts Neues; Seeds-only-Modus umgeht den Store gezielt.
17.09.	Zusammenfassung des Deep Runs (`summaries/`) nicht in S3	Task-Rolle ohne Schreibrecht auf das Präfix; Per-Hochschul-Diagnose des Laufs ging verloren (aus den Manifesten rekonstruierbar).
16.09.	Evaluation überschreibt LLM-Urteile (Fehler im Code)	Alle bisherigen LLM-Ergebnisse wurden stillschweigend ignoriert; Korrektur in `rescore_decisions`; ehrliche Abdeckung sank von 314 auf 303.

Was beim ersten Durchlauf des Bootstrap-Skripts schiefging. Der Neuaufbau am 12. August lief nicht auf Anhieb durch, und jede Panne ist im Skript heute abgefangen:

- *PowerShell-Fehlerbehandlung:* Mit `$ErrorActionPreference = SStop"` bricht Windows PowerShell 5.1 bei *jeder* stderr-Ausgabe eines nativen Programms ab, auch bei Exit-Code 0. Phase 8 stoppte deshalb ohne echten Fehler; die Fehlerprüfung läuft seitdem explizit über Exit-Codes.
- *Elastic IP nicht verknüpft:* Das Zuweisen der Elastic IP schlug fehl, weil die Instanz noch nicht bereit war; wegen des geänderten Fehlermodus blieb das unbemerkt. Die Webapp war unter der Elastic IP nicht erreichbar, bis die Zuordnung von Hand nachgeholt wurde. Das Skript wartet nun auf `instance-running` und prüft die Zuordnung mit Wiederholungen.
- *Fargate-Cluster fehlte:* Beim ersten Bulk-Start meldete die Webapp `ClusterNotFoundException`, weil die Fargate-Phase nicht vollständig durchgelaufen war. Ein Teil-Durchlauf (`-SkipImage -SkipEc2 -SkipCicd`) holte sie nach, was die Idempotenz bestätigt.

Die Budgetsperre im Detail. Nach rund zwei Wochen Abwesenheit war der Pool eingefroren. Die Abrechnung (Abb. 5.2) zeigt die Ursache: Die erste LLM-Prüfung am 2. September kostete 21,61 $ an einem einzigen Tag und brachte die kumulierten Kosten auf 48 $; zusammen mit der Dauerlast der Webapp war die 50-Dollar-Grenze am 05.09. überschritten. Eine Prüfung aller Regionen ergab keinen aktiven Verursacher im Sinne eines hängenden Tasks. Als Konsequenz wurden die LLM-Aufrufe zunächst ausgesetzt, und es wurde eine Terminierungsgarantie eingebaut, die ein „Hängen“ als Kostenursache ausschließt (Abschnitt 5.8). Nach der Freigabe des Pools liefen die LLM-Prüfungen der letzten Septembertage (30.09.–02.10.) mit zusammen rund 87 $ an drei Tagen.

5.8 Kosten und Kostenkontrolle

Die Kosten stammen aus der Abrechnung der Cloud-Umgebung (Juli bis 5. Oktober 2026) und betragen insgesamt 155,60 $ (Abb. 5.1). Der Cost Explorer ist per SCP für die Studierendenkonten gesperrt; die Werte stammen aus dem exportierten Kostenbericht. Pro gefundenem Modulhandbuch (geschätzt rund 18.628) entspricht das weniger als einem Cent, einschließlich Entwicklung, Tests und Webapp-Betrieb.

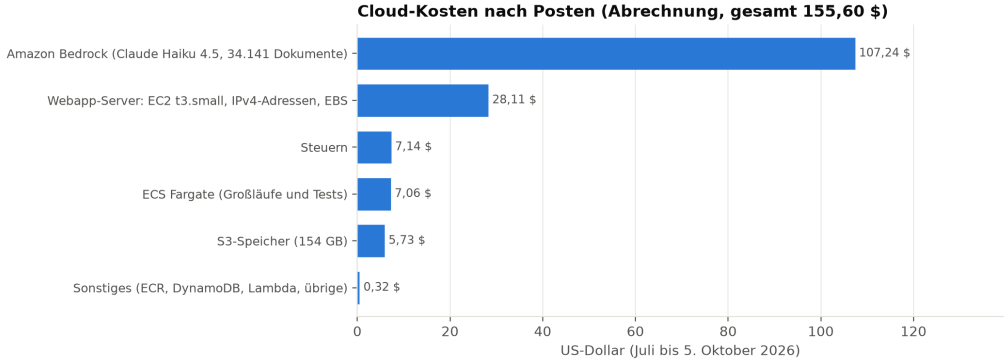


Abbildung 5.1: Cloud-Kosten nach Posten (Abrechnung, Juli bis 5. Oktober 2026).

Bemerkenswert ist die Rangfolge: Nicht der Crawl dominiert die Kosten, sondern die *Qualitätssicherung* durch das LLM (107 $, rund 69 %) und der dauerhaft laufende Webapp-Server samt IPv4-Adressen und Datenträger (28 $). Die Rechenzeit der Fargate-Läufe liegt insgesamt bei

nur 7 $ (der August-Lauf kostete rund 3,70 $, der Deep Run rund 3,30 $), der S3-Speicher bei 6 $. Das bestätigt die Grundidee des Aufbaus, Rechenlast kurzlebig und Dauerbetrieb klein zu halten, und zeigt zugleich, dass eine Dauerressource im Budget des Sandbox-Pools relevanter ist als jeder Crawl. Die LLM-Prüfung kostete rund 3,10 $ je 1.000 Dokumente; eine Prüfung *aller* Dokumente wäre auf rund 500 $ gekommen.

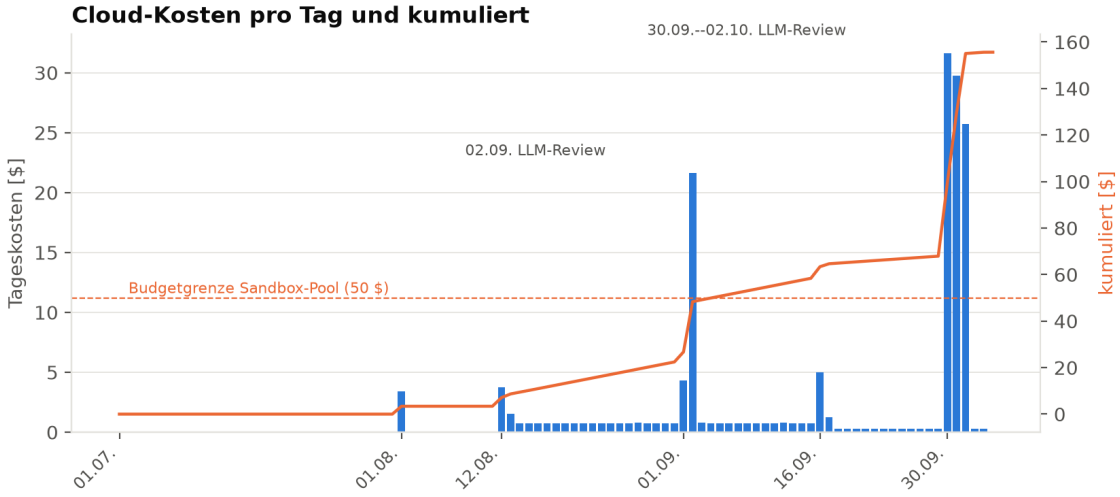


Abbildung 5.2: Tageskosten und kumulierte Kosten. Die Spitzen entsprechen den Großläufen (12.08., 16.09.) und den Tagen der LLM-Prüfung (02.09., 30.09.–02.10.); die Grundlast ist die dauerhaft laufende Webapp. Die gestrichelte Linie markiert die Budgetgrenze von 50 $.

Terminierungsgarantie. Die Budgetsperre führte zu einem dreischichtigen Schutz gegen unbegrenzte Laufzeit: (1) `DOWNLOAD_TIMEOUT` von 60 s je Anfrage, (2) `CLOSESPIDER_TIMEOUT` als Zeitlimit je Hochschule und (3) ein Wachhund (`bulk_run._arm_runtime_watchdog`), ein Daemon-Timer, der nach `BULK_MAX_RUNTIME_SECONDS` den Prozess mit `os._exit(2)` beendet, auch wenn der Reactor hängt. Dadurch endet der ECS-Task und die Abrechnung. Die Funktion wurde getestet und feuert zuverlässig.

5.9 Lehren aus dem AWS-Betrieb

1. **Reproduzierbarkeit schlägt Eleganz.** Das Bootstrap-Skript aus AWS-CLI-Befehlen war robuster als der „saubere“ Terraform-Stack, weil es mit einem unbekannten Zielkonto umgehen konnte.
2. **Daten sind wertvoller als Infrastruktur.** Infrastruktur lässt sich neu bauen, Labels und Zwischenergebnisse nicht. Wertvolles gehört lokal gesichert.
3. **Dauerressourcen zählen.** Im Budgetrahmen einer Sandbox ist eine immer laufende Kleininstanz gewichtiger als ein zwölfstündiger Großlauf.
4. **Zugriffsbeschränkungen sind Entwurfsvorgaben.** SCP-Sperren (Bedrock-Mantle, Cost Explorer), kurzlebige Schlüssel und TLS-Proxy erzwangen wiederaufnehmbare Verarbeitung in kleinen Blöcken.
5. **Gerechnet ist nicht gemessen.** Die Lambda-Rechnung des MVPs („zehn Minuten“) war für das Zielbild um etwa den Faktor 40 zu optimistisch; erst Messungen am Großlauf haben die richtige Laufzeitklasse gezeigt.

6 Der Crawler: Entwicklung und Recall-Arbeit

Der Crawler ist das Herzstück des Systems und zugleich das, was sich am häufigsten geändert hat. Dieses Kapitel folgt dem Weg von einem einfachen Tiefen-Crawl zu einer gesteuerten, seeds-gestützten Breitensuche. Es nennt zu jedem Schritt das Problem, die Maßnahme und die Wirkung, und es beschreibt ausdrücklich, was nicht funktioniert hat.

6.1 Ausgangspunkt: ungesteuerter Tiefen-Crawl

Der erste Spider folgte allen Links derselben Domain bis zu einer festen Tiefe (zunächst 2) und einer Seitengrenze (60) und lud PDF- und Word-Dateien herunter. Die Hochschulliste enthält nur Startseiten, daher wurde ein begrenzter Tiefen-Crawl nötig (`CRAWL_MAX_DEPTH`, `CRAWL_MAX_PAGES`, `allowed_domains`). Im Juli zeigte ein Test auf der TU Dortmund das Grundproblem: Von der Startseite aus wurden 934 Dokumente gefunden, 372 heruntergeladen und *nur 4* waren echte Modulhandbücher, bei rund 156 Studiengängen. Die Ursache war doppelt: Der Crawl folgte Links in beliebiger Reihenfolge und verbrauchte das kleine Seitenbudget für Nachrichten und Impressum, und Modulhandbücher liegen auf *Fakultäts-Subdomains* (etwa `cs.tu-dortmund.de`) in Tiefen, die das Budget nie erreichte. Das Durchprobieren von Tiefe 1–3 und 25–200 Seiten in Lambda führte zudem ins Zeitlimit (Kapitel 5).

6.2 Fokussierter Crawl (24. Juli)

Die erste größere Verbesserung bestand aus drei Maßnahmen:

1. **Best-First-Frontier.** Jeder Link wird nach Schlüsselwörtern in URL und Ankertext bewertet (Umlaute gefaltet); die Bewertung wird zur Scrapy-Request-Priorität, Dokumente erhalten einen Grundbonus. Das Wortlisten-Vokabular (`modulhandbuch`, `modulbeschreibung`, `studiengang` usw. positiv; `impressum`, `news` usw. negativ) liegt im Profil.
2. **Sitemap-Seeding.** `start_requests` lädt `/sitemap.xml` sowie die in der robots.txt genannten Sitemaps, behandelt Gzip und Index-Rekursion und lädt passende Dokumente direkt.
3. **Subdomain-Erkennung.** Springt ein Link in eine neue Subdomain derselben Basisdomain, wird die Tiefe zurückgesetzt, die Priorität erhöht und die Sitemap der Subdomain geladen (begrenzt über `CRAWL_MAX_SUBDOMAIN_SITEMAPS`).

Am selben Tag wurde die Engine **modularisiert**: Linkbewertung, Zieldefinition und Extraktion liegen in austauschbaren *Extraktionsprofilen* (Kapitel 4). Außerdem wurden **skriptgenerierte PDFs** ohne `.pdf` in der URL erkannt (RWTH: `show_document.asp?id=...`, `download.php?id=...`): Die Antwort wird anhand des Content-Types als Dokument erkannt und der Dateiname mit einem Hash versehen, damit sich Skript-URLs in S3 nicht überschreiben.

6.2.1 Rauchtest auf vier Hochschulen

Ein Test auf vier Hochschulen (769 PDFs; 202 positiv, 41 Review, 526 negativ) zeigte, dass die Wirksamkeit stark von der Seitenstruktur abhängt (Tabelle 6.1). Die Mechanismen funktionierten, der Recall ist *strukturabhängig*: nahezu vollständig bei einfachen Seiten, teilweise bei Riesenuniversitäten und null bei JavaScript-Startseiten. Diese Erkenntnis bestimmte die weitere Arbeit.

Tabelle 6.1: Rauchtest auf vier Hochschulen (24. Juli).

Hochschule	**Ergebnis**	**Beobachtung**
FH Wedel	sehr gut	Informatik-Handbuch mit Score 0,96 gefunden
TU Dortmund	teilweise	8 Fakultäts-Subdomains erkannt, Informatik-Handbuch gefunden, aber nur etwa 28 von 156 Studiengängen (Budget)
RWTH Aachen	teilweise	skriptgenerierte PDFs erkannt (Review-Band um 0,6), die Informatik-Fakultät aber nicht erreicht
FH Münster	**null**	JavaScript-Startseite ohne Sitemap: Scrapy sieht keine Links

6.3 Großlauf über die Hochschulliste (12. August)

Der erste Großlauf über die Hochschulliste lag im verlorenen Sandbox-Pool (Kapitel 5) und wurde am 12. August in der neu aufgebauten Umgebung wiederholt; alle folgenden Zahlen stammen aus dieser Wiederholung.

Der Großlauf über 411 Listeneinträge (404 Hochschulen nach Zusammenführung von Einträgen mit gleicher Domain) lief 6,3 h auf einem Fargate-Task mit 4 vCPU und 16 GB und lud 65.944 PDFs herunter (Mittel rund 10.467 PDFs pro Stunde, Median 285 s je Hochschule). Die Evaluation (eigenes Auswertungspaket mit HTML-Bericht, Szenarien nach Typ, Träger, Größe und Bundesland) ergab:

- 11.579 automatisch positive, 8.166 Review-Fälle und 46.199 negative Dokumente;
- Abdeckung nur 261 von 404 Hochschulen, **145 Hochschulen ohne ein einziges Handbuch**, damit lag die Zahl der Treffer bei knapp 30 % des Erwartungswerts von rund 40.000;
- Die Lücke war konzentriert auf kleine und nicht-öffentliche Hochschulen: öffentlich 79 %, privat 35 %, kirchlich 49 % Abdeckung; nach Größe von 49 % (unter 1.000 Studierende) bis 92 % (15.000–30.000);
- NORDAKADEMIE, bei der erwartungsgemäß keine Handbücher online liegen, lieferte 9 „positive“ Treffer; ein Hinweis auf Fehlalarme, der sich später bestätigte (Kapitel 7);
- Die FernUniversität Hagen, mit Abstand die größte Hochschule Deutschlands, lieferte nur 57 Positive und 145 Review-Fälle; ein Hinweis auf zu geringe Tiefe pro Hochschule (siehe Phase 2 unten).

Die Diagnose zerlegte das Problem in drei unabhängige Lücken: (A) 82 Hochschulen wurden gecrawlt, hatten aber null Handbücher, (B) 70 Hochschulen lieferten null Dokumente, und (C) auch bei Hochschulen mit Treffern war der Recall pro Hochschule gering. Stichproben zeigten, dass die als „leer“ gemeldeten Hochschulen (Konstanz, FH Kiel) sehr wohl

Handbücher an erratbaren URLs haben (`/uploads/tx_studiengang/modulebook_*.pdf`, `/fileadmin/.../modulhandbuch_*.pdf`): Nicht die Abwesenheit, sondern *Auffindbarkeit* war der Engpass.

6.4 Schnelle Gewinne ohne Zusatzbudget

Zwei Anpassungen kosteten nichts:

- **Schwelle:** Das ausgelieferte Modell hatte eine obere Schwelle von 0,65. Ein Neubewerten der *bereits heruntergeladenen* 65.944 Dokumente bei 0,50 ergab +2.333 Positive (11.579 → 13.912) und +10 Hochschulen (261 → 271), bei rund 88 % Precision auf dem Hold-out. Ein „What-if"-Schalter der Evaluation (`-upper`) rechnet Entscheidungen aus gespeicherten Scores neu, ohne Neu-Crawl. (Dass diese Absenkung später zur Positiv-Flut beitrug, ist Teil von Kapitel 7.)
- **Vokabular:** Dateinamenmuster, die vorher ohne Gewicht blieben (`modulebook`, `mhb`, `handbuch`, `spo`, `studienverlaufsplan`), wurden ins Profil aufgenommen, sodass Best-First sie früher erreicht.

6.5 Discovery: Seeds aus Suchindizes

Dominanter Hebel war die *Discovery*: Pro Hochschule wird vorab gesucht, wo Modulhandbücher liegen, und diese URLs werden als priorisierte Ziele in den Crawl eingespeist (`DISCOVERY_SEEDS_PATH`, in der Cloud über S3). Tabelle 6.2 zeigt alle erprobten Anbieter, auch die verworfenen.

Wirkung. Ein Test auf drei Hochschulen, die im ersten Lauf null Treffer hatten (htwsaar, Konstanz, FH Kiel), ergab mit Serper und Common Crawl nach einem kurzen Crawl 42 klassifizierte Dokumente, davon 7 automatisch positive; die LLM-Zweitstufe bestätigte weitere 6 englische Handbücher, die das TF-IDF-Modell unsicher beurteilt hatte. Zusammen 13 Handbücher bei vorher null.

6.6 Seeds-only-Modus: Lücken ohne Duplikate schließen

Der Visited-Store in DynamoDB dedupliziert auf Ebene der *Hochschule* (Start-URL), nicht des Dokuments: Ein erneuter Lauf mit Discovery übersprang jede bereits gecrawlte Hochschule, und die Discovery lief nie. Ein vollständiges Leeren der Tabelle hätte alle Hochschulen neu gecrawlt (und zu physischen Duplikaten in S3 geführt, da der S3-Schlüssel die Job-ID enthält). Die Lösung war ein **Seeds-only-Modus**: Der Spider holt ausschließlich die Seeds, überspringt den Visited-Store, und Hochschulen ohne Seeds werden sofort übergangen. Der erste Lauf (3. September) holte 431 PDFs von 134 Hosts (226 positiv, 179 negativ, 26 Review) und hob die Abdeckung *in wenigen Minuten* von 271 auf 314 Hochschulen (+43), die Positive von 13.912 auf 14.133. Das bewies den Ansatz.

Tabelle 6.2: Erprobte Discovery-Quellen.

Quelle	Status	Befund
Common Crawl (CDX)	**genutzt**	Kostenlos, ohne Schlüssel; je Domain PDFs mit „modul/handbuch“-Mustern. Von 18 getesteten Problemdomänen wurden für 11 Handbücher gefunden; stark bei Fakultäts- und Modul-Datenbank-Subdomains. Blind für tief verlinkte, unverlinkte Dateien. Mehrere Monatsindizes werden vereinigt, Anfragen mit Backoff und Pausen (CC drosselt Bursts und sperrte die Test-IP zeitweise).
Serper (Google-SERP)	**genutzt**	Abfrage `site:<basis> <begriff>`; findet tiefe Dateien wie das Konstanzer `modulebook`. Die kostenlose Stufe verlangt Anpassungen (kein `filetype:`, höchstens 10 Treffer je Anfrage); eine bezahlte Stufe war nicht möglich.
Firecrawl	vorbereitet, kaum getestet	Rendert JavaScript, liefert Link-Karten und ersetzt eine Seite durch ihr gerendertes HTML (Middleware). Für IU hat die Karte *keine* PDF-Handbücher gefunden, das Portal ist ein reiner JavaScript-Katalog. Die Pipeline für Katalog-Seiten (Inhaltsklassifikation statt PDF) existiert, wurde aber nicht im Großlauf eingesetzt.
Google Custom Search	**verworfen**	Google hat „gesamtes Web durchsuchen“ aus der Programmable Search Engine entfernt; nur noch 50 gelistete Domains, damit für 400 Hochschulen nutzlos.
DuckDuckGo	**verworfen**	Aus der Cloud-Umgebung CAPTCHA-gesperrt.
Sitemaps allein	unzureichend	Tiefe Dateien (z. B. Konstanz) stehen in keiner Sitemap; die Universität hat keine `sitemap.xml`.
Brave Search API	nicht gebaut	Unabhängiger Index, wäre die einzige realistische Quelle für die tiefsten Dateien, benötigt aber Anmeldung und eventuell Zahlungsmittel.

6.6.1 robots.txt und Seeds

Bei elf Hochschulen blieb der Seeds-Lauf zunächst bei null heruntergeladenen Dateien, obwohl Seeds vorlagen. Die Vermutung war JavaScript oder robots.txt; tatsächlich lag es an einer veralteten Seed-Datei im Lauf. Das echte robots-Problem trat bei der Universität Konstanz auf (`Disallow: /`). Die Lösung ist `SEEDS_BYPASS_ROBOTS` (standardmäßig *aus*): Es setzt `dont_obey_robotstxt` *nur* auf Requests für Discovery-Seeds, also auf einzelne, vom Suchindex als öffentlich ausgewiesene Handbuch-URLs, während der Link-Crawl der Seite robots.txt weiter respektiert. Im Test lieferte Konstanz damit 13 von 18 Dateien (6 MB). Diese Ausnahme ist eine bewusste Abwägung, die im Ausblick und in der Diskussion (Kapitel 9) aufgegriffen wird.

6.7 Phase 2: Vollständigkeit je Hochschule

Auch bei Hochschulen mit Treffern blieb der Recall niedrig. Beispiel FernUniversität Hagen: nur 70 Handbücher, erwartet waren hunderte. Die Ursache: Der gesteuerte Crawl *tunnelte* in die erste starke Fakultät (`/mi/`) und erreichte nie Fakultäten, deren Modulkataloge hinter neutraler Navigation liegen (`/psychologie/studium/portale/<abschluss>/...`, kein „modul“ im Pfad). Gegenmaßnahmen:

- **Breadth-Fairness:** `CRAWL_MAX_PAGES_PER_SECTION` begrenzt die Seitenfolgen pro erstem Pfadsegment (Fakultät) und erzwingt Streuung; Dokument-Downloads werden nie gekappt.
- **Navigations-Tokens** im Profil (`portal`, `bsc`, `msc`, `studienangebot` usw.), damit schlüsselwortlose Programm-Zweige verfolgt werden.
- **Deep-Run-Konfiguration:** Tiefe 5, 2.500 Seiten, 60 Seiten pro Sektion, bis 3.000 Seiten und 1.800 s je Hochschule, Gesamtlaufzeit bis 12 h, `force`.
- **Terminierungsgarantie** und Diagnose-Statistiken je Hochschule (Abbruchgrund, Seitenzahl, Sektionsverteilung).

Der lokale Test an Hagen zeigte, dass der Crawl jetzt Wirtschaft, Psychologie ($0 \rightarrow 4$ Positive) und Kultur- und Sozialwissenschaften erreichte statt nur einer Fakultät.

6.7.1 Deep Run (16./17. September)

Der Deep Run lief 12,0 h und lud 132.193 PDFs herunter (37.993 automatisch positiv für sich genommen). Hagen stieg von 70 auf **870** Positive (Faktor 12), verteilt über sieben Fakultäten. Die kumulierte Vereinigung aller Läufe (Aug-Lauf, Seeds-Läufe, Deep Run) ergab 164.396 Dokumente, 43.270 TF-IDF-Positive bei Schwelle 0,5 und eine Abdeckung von 321 Hochschulen. Das erreichte *nominell* das 40.000er-Ziel, war aber durch die Precision-Probleme noch nicht belastbar (Kapitel 7).

6.8 Höflichkeit und Grenzen

Der Crawler respektiert robots.txt, drosselt (Basiswartezeit 1 s plus AutoThrottle) und tritt mit identifizierbarem User-Agent auf. Weitere harte Grenzen (Dateigröße 50 MB, Timeout 60 s je Anfrage) sind in Anhang A aufgeführt. Weiterhin außerhalb der Reichweite sind Handbücher hinter Login, nur zur Laufzeit erzeugte Dokumente ohne Datei und Handbücher auf einer *anderen* registrierbaren Domain als der Startseite (`_same_site` vergleicht die letzten zwei DNS-Labels).

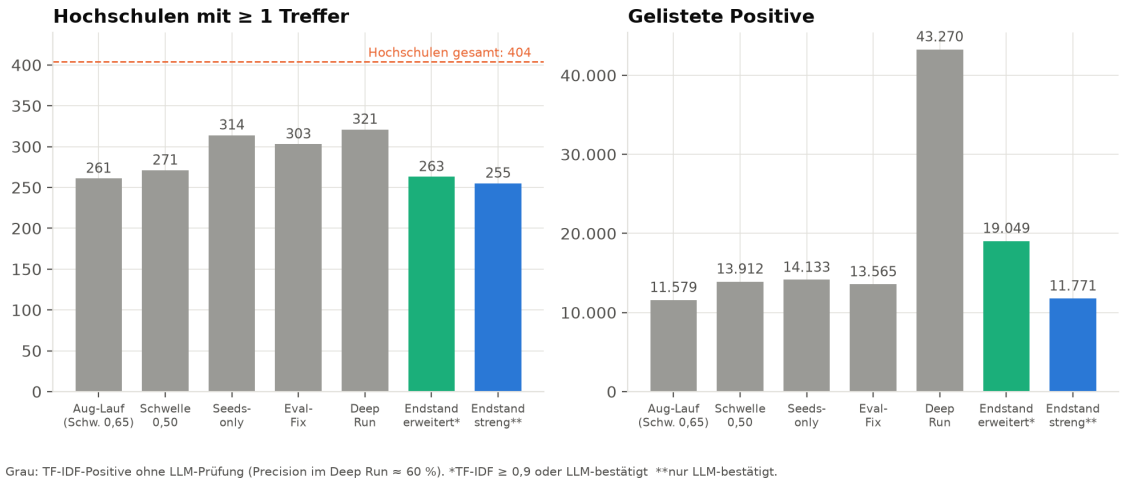


Abbildung 6.1: Abdeckung (links) und gelistete Positive (rechts) im Projektverlauf. Die grauen Balken sind TF-IDF-Positive ohne LLM-Prüfung; erst die Endstände (grün, blau) sind qualitätsgesichert.

6.9 Zusammenfassung der Crawl-Entwicklung

Tabelle 6.3 bündelt den Weg.

Tabelle 6.3: Crawl-Strategien im Zeitverlauf mit Problem, Maßnahme und Wirkung.

Zeitpunkt	Problem	Maßnahme und Wirkung
Juli	Budget für Unwichtiges verbraucht, TU Dortmund 4 Treffer in 934 Dokumenten	Best-First, Sitemap, Subdomains; Handbuch-Funde deutlich höher
24.07.	Skript-URLs, JS-Startseiten	Content-Type-Erkennung (RWTH gelöst); JS-Startseiten (FH Münster) *ungelöst*
12.08.	145 Hochschulen ohne Treffer	Auswertungsmodul; Schwelle 0,5: +2.333 Positive, +10 Hochschulen
27.08.	Handbücher nicht auffindbar	Discovery (Common Crawl, Serper), OCR, LLM-Zweitstufe
03.09.	Visited-Store blockiert Wiederholung	Seeds-only: +43 Hochschulen in Minuten
16.09.	robots-Sperre (Konstanz), Tunneln (Hagen)	Seeds-Robots-Bypass; Breadth-Fairness; Deep Run: Hagen 70 → 870

7 Klassifikation: vom TF-IDF-Modell zur LLM-Zweitstufe

Dieses Kapitel ist der Kern des Machine-Learning-Anteils. Es zeigt, wie ein auf dem Papier sehr gutes Modell im realen Crawl deutlich schlechter abschnitt, wie das erkannt wurde und welche Maßnahmen die Qualität schließlich auf ein belastbares Niveau gebracht haben.

7.1 Aufgabe und Daten

Die Aufgabe ist binär: *Ist dieses PDF ein Modulhandbuch?* Als Modulhandbuch zählt (Definition im Prompt der Zweitstufe) ein systematisches Handbuch, das die Module eines Studiengangs beschreibt. Einzelne Modulblätter, Prüfungsordnungen, Studienverlaufspläne und Vorlesungsverzeichnisse zählen *nicht*. Diese strenge Definition ist eine Entscheidung; sie ist der Hauptgrund für viele Beinahe-Treffer.

Die Trainingsdaten (`modulhandbuecher/{positiv,negativ}/<hochschule>/`) wuchsen in drei Stufen (Tabelle 7.1):

- **MVP-Modell (Juli):** 202 Dokumente (134 positiv, 68 negativ) aus 21 Hochschulen nach Deduplizierung per normalisiertem Text-Hash vor dem Split.
- **Nach Feedback (22. Juli):** Die Review-UI lieferte 157 Verdikte (9 positiv, 148 negativ), das nachtrainierte Modell nutzte 356 Dokumente aus 22 Hochschulen (143 positiv, 213 negativ). Die Verdikte waren vor allem harte Negative, das heißt Dokumente, die dem Modell zuvor als Handbuch erschienen.
- **Nach der Massenerhebung (3. September):** Mit über 5.000 zusätzlichen Handbüchern aus dem Crawl wuchs der Datensatz auf **2.695 Dokumente** aus **269 Hochschulen** (1.971 positiv, 724 negativ).

7.2 Modell und Evaluation

Das Modell ist eine logistische Regression auf TF-IDF-Merkmalen (Wort- und Zeichen-n-Gramme) des extrahierten PDF-Texts. Die Evaluation nutzt *StratifiedGroupKFold, gruppiert nach Hochschule*: Keine Hochschule steht gleichzeitig im Training und im Test, es wird also die Erkennung an *ungesehenen* Hochschulen gemessen (Szenario B). Ein Leakage-Check trainiert zwei Varianten, *nur Inhalt* und *Inhalt plus Dateiname/Metadaten*; sind sie ähnlich gut, liest das Modell das Dokument und nicht den Dateinamen. Die Schwellen werden auf 95 % Recall kalibriert (Kapitel 3).

Die Leakage-Prüfung bestätigte das Modell: Inhalt und Inhalt-plus-Metadaten liegen nahe beieinander (MVP 0,957 gegenüber 0,960; September sogar leicht schlechter mit Metadaten), also wird das schlankere Inhaltsmodell ausgeliefert. Die Konfusionsmatrix der September-Variante am Recall-Ziel: 1.873 richtig positive, 98 falsch negative, 656 richtig negative, 68 falsch positive. Anhang B enthält die Details.

Tabelle 7.1: Modellversionen und gruppierte Kreuzvalidierung.

Version	Dok.	Hochsch.	PR-AUC	ROC-AUC	Prec.	Recall
MVP (Juli), Variante *content*	202	21	0,957	0,941	0,871	0,955
MVP, *content + Dateiname*	202	21	0,960	0,926	0,848	0,955
Stand September, *content*	2.695	269	0,988	0,973	0,965	0,950
September, *content + Metadaten*	2.695	269	0,976	0,959	0,958	0,950

Precision und Recall am Schwellenwert für das Recall-Ziel von 95 % (MVP: 0,387; September: 0,627). Beim Schwellenwert 0,5 erreicht die September-Variante *content* Precision 0,952 und Recall 0,985.

7.3 Der Praxis-Gap: 96,5 % im Training, 25 % im Crawl

Die Trainingsmetriken waren hervorragend, doch im realen Crawl verhielt sich das Modell anders. Zwei Beobachtungen kamen aus dem Betrieb:

1. NORDAKADEMIE lieferte neun „Handbücher“, obwohl dort keine online sind.
2. Die LLM-Verifikation der 226 automatisch positiven Treffer aus dem Seeds-Lauf (215 prüfbar) ergab nur **129 echte Handbücher** und 86 Fehlalarme (rund 40 %): Studienordnungen, Verkündungsblätter, ein Bewerbungsformular mit Score 0,79, Lehrbücher.

Die systematische Auswertung nach Score-Bereichen (Abb. 7.1) zeigt, wie stark der Score überschätzt: Bei Score 0,5–0,6 bestätigt das LLM nur 8 % der Treffer, bei 0,8–0,9 sind es 44 %; erst oberhalb von 0,9 liegt die Bestätigungsrate bei 82,5 % (Pilotstichprobe, $n = 400$, Wilson-Intervall 78,5–85,9 %).

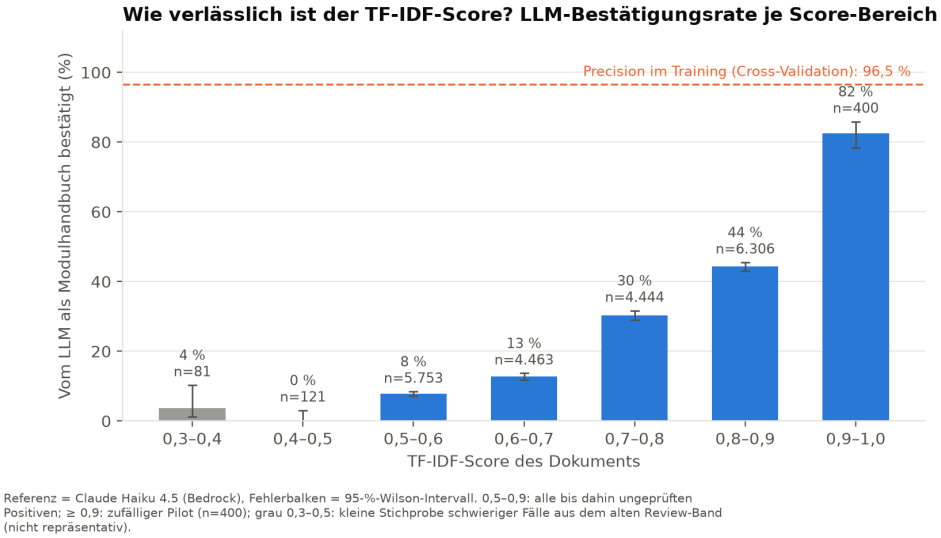


Abbildung 7.1: Anteil der vom LLM bestätigten TF-IDF-Positive je Score-Bereich (LLM als Referenz). Die Bereiche oberhalb 0,9 stammen aus der Pilotstichprobe.

Ursachen. (1) Das Trainingsset bildet die Breite der Beinahe-Treffer im Web nicht ab: Die Negativbeispiele waren überwiegend leichte Fälle, während der Crawl Zehntausende Prüfungsordnungen, Modulblätter und Studienverlaufspläne enthält. (2) Die Schwelle wurde im Deep Run auf 0,5 gesenkt, um Recall zu gewinnen, das aktuelle Modell schlägt 0,8 vor; das erzeugte die Positiv-Flut (37.993 Positive allein im Deep Run). (3) Die Verteilung unterscheidet sich

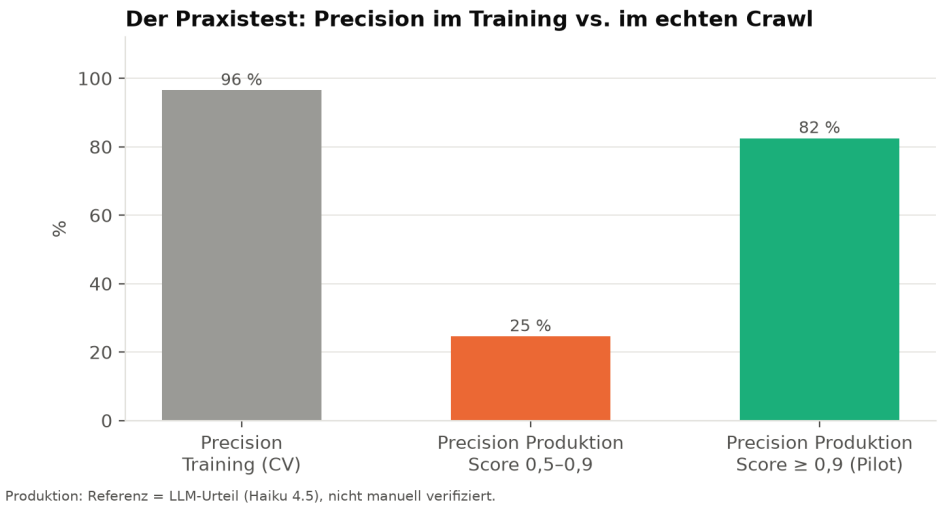


Abbildung 7.2: Precision im Training (gruppierte Kreuzvalidierung) gegenüber dem realen Crawl. Die Differenz ist die Kernaussage zur Übertragbarkeit.

zwischen Hochschulen mit Handbuch-Datenbank (viele ähnliche Dokumente je Host) und den Trainingshochschulen.

Welche Dokumente werden verwechselt? Die LLM-Begründungen der 20.031 verworfenen TF-IDF-Positive wurden per Stichwort in Fehlerarten eingeteilt (Abb. 7.3): die Hälfte sind Prüfungs- oder Studienordnungen (50 %), 21 % einzelne Modulblätter oder Syllabi, 10 % Studienverlaufspläne, 5 % unvollständige Modulkataloge, 4 % Veranstaltungsverzeichnisse und 2 % Formulare. Das sind genau die Dokumenttypen, die dem Modulhandbuch semantisch am nächsten liegen. Die Zuordnung ist stichwortbasiert und daher grob.

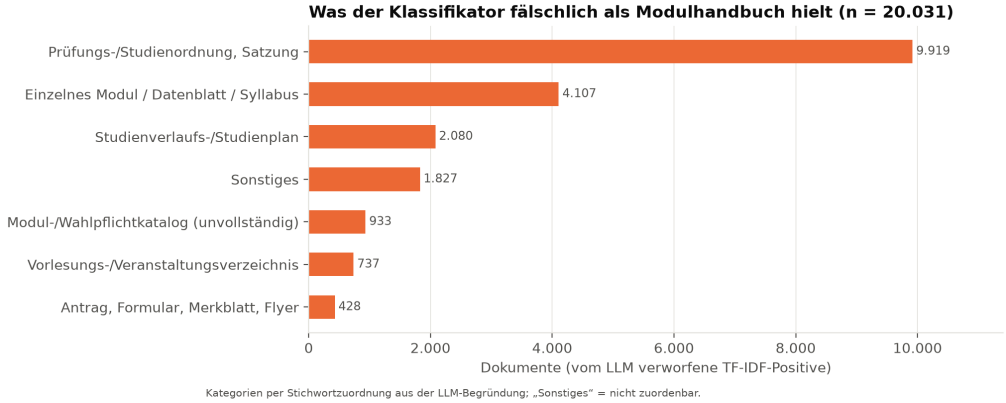


Abbildung 7.3: Fehlerarten der TF-IDF-Positive, die das LLM verworfen hat.

Dateiname als Signal. Obwohl das Modell nicht auf Dateinamen trainiert wurde, ist der Dateiname ein starkes Zusatzsignal (Abb. 7.4): Enthält er „modul", „handbuch" oder „mhb", werden bei Score $\geq 0{,}9$ 95 % bestätigt ($n = 4.399$), ohne diese Stichwörter nur 35 % ($n = 3.529$); im Bereich 0,5–0,7 sind es 36 % gegenüber 8 %. Das ist ein Ansatzpunkt für ein künftiges Modell (Kapitel 10).

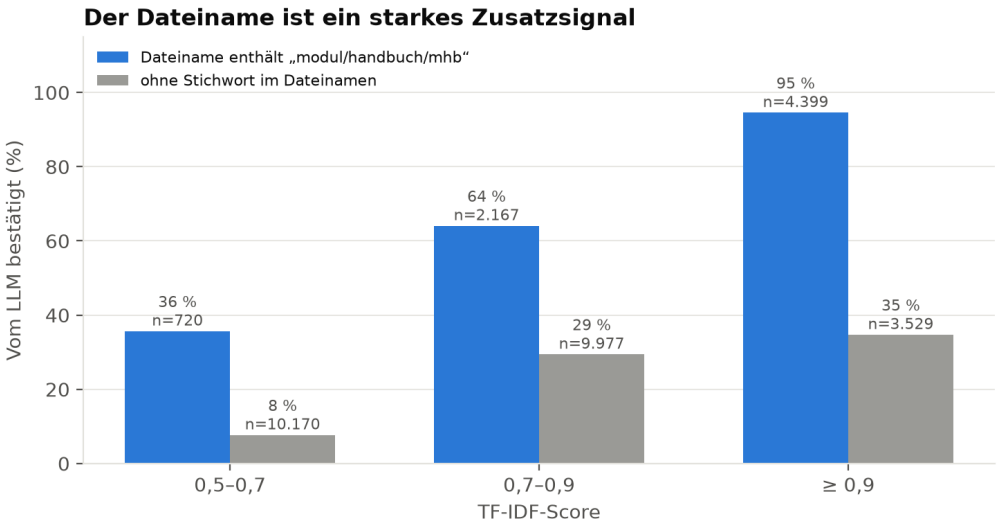


Abbildung 7.4: Bestätigungsrate des LLM mit und ohne Handbuch-Stichwort im Dateinamen.

7.4 Human-in-the-Loop

Die erste Antwort auf Modellfehler war ein *Menschen-Review*. Die Review-UI der Webapp zeigt relevante Dokumente (Positive und Review-Band) nach Score sortiert, mit Seitenweise-Anzeige für große Mengen und Download. Verdikte werden gesammelt (nicht einzeln) an S3 geschrieben, über das Kommando `feedback-retrain -from-s3` geholt, die Dokumente unter dem *menschlichen* Label in den Trainingsbaum einsortiert (nach Hochschule gruppiert) und das Modell neu trainiert; das neue Modell wird per Git eingecheckt und über CI/CD ausgeliefert. Der Mechanismus ist modellunabhängig (positiv/negativ), nicht modulhandbuchspezifisch. In der Praxis stieß der manuelle Weg an die Skala: Bei zehntausenden Kandidaten ist menschliches Prüfen nicht tragfähig; das war der Anlass für die LLM-Zweitstufe.

7.5 LLM-Zweitstufe

Aufbau. Die Zweitstufe (`mlclassifier/llm_review.py`) liest die Manifest-Einträge eines Bandes (Review, automatisch positiv oder ein Score-Fenster), lädt die PDFs aus S3, extrahiert Text, sendet die ersten 8.000 Zeichen des Textes an Claude Haiku 4.5 auf Amazon Bedrock und erhält ein strukturiertes Urteil mit Begründung und Konfidenz zurück. Der Prompt wird aus einer `ClassificationSpec` gebaut (Definition, Beispiele, Ausschlüsse), die Zweitstufe ist also wiederverwendbar. Ein Urteil gilt nur bei Konfidenz $\geq 0{,}8$ als bestätigt (praktisch liegen alle Urteile zwischen 0,8 und 1,0). Die Verarbeitung läuft parallel (Thread-Pool, standardmäßig 12 Arbeiter, mit Sperre für Manifestschreiben und fehlerisolierten Einzelaufrufen); das senkte die Dauer für rund 6.000–8.000 Dokumente von etwa fünf Stunden auf etwa 30 Minuten. Das Modell wurde bewusst klein gewählt (Haiku): billig genug für zehntausende Dokumente, im Test aber hinreichend genau (6 von 6 Zweifelsfällen aus htwsaar mit Konfidenz 0,95 korrekt als Handbuch erkannt).

Läufe. Tabelle 7.2 listet die Läufe.

Tabelle 7.2: LLM-Läufe der Zweitstufe.

Lauf	geprüft	bestätigt	Rate
Review-Band des Aug-Laufs (Score 0,3–0,7)	6.065	1.845	30 %
Positive 0,5–0,9 des Deep Runs	20.966	5.153	25 %
Pilot: Stichprobe der Positive ≥ 0,9	400	330	82,5 %
OCR-Treffer (Score ≥ 0,5)	247	42	17 %
Positive ≥ 0,9 auf 197 verdächtigen Hosts	6.463	4.404	68 %
Summe (je URL eindeutig)	**34.141**	**11.771**	

Die Pilotstichprobe. Um nicht alle rund 13.700 Dokumente oberhalb von 0,9 mit dem LLM prüfen zu müssen (rund 35 $), wurde zunächst eine Zufallsstichprobe von 400 geprüft. 330 wurden bestätigt (82,5 %, Wilson-Intervall 78,5–85,9 %). Das lag unter dem erhofften Wert von 95 %.

Der Host-Effekt. Die Aufschlüsselung der LLM-Urteile nach Host zeigte die Ursache: Auf 197 Hosts, auf denen das LLM bei den übrigen Dokumenten unter 30 % bestätigt hatte, waren selbst Dokumente mit Score ≥ 0,9 nur zu rund 68 % echt, auf allen anderen Hosts dagegen zu 94,2 % (Abb. 7.5). Die Qualität hängt also nicht nur vom Score ab, sondern stark vom Host (typisch sind Server, die vor allem Prüfungsordnungen und Modulblätter ablegen). Deshalb wurden gezielt die 6.463 Positiven dieser Hosts vom LLM geprüft (4.404 bestätigt) und der Rest durch eine zweite Stichprobe auf den übrigen Hosts bewertet: 212 von 225 bestätigt (94,2 %, Wilson-Intervall 90,4–96,6 %). Daraus folgt eine *Schätzung* von rund 18.628 echten Modulhandbüchern unter den 19.049 Dokumenten der erweiterten Liste (Bereich 18.348–18.801), also eine Precision von rund 98 % *gemessen am LLM-Urteil*. Die gezielte Nachprüfung hob die Precision der Endliste damit von etwa 89 % auf etwa 98 %, ohne alle ungeprüften Dokumente zu bewerten.

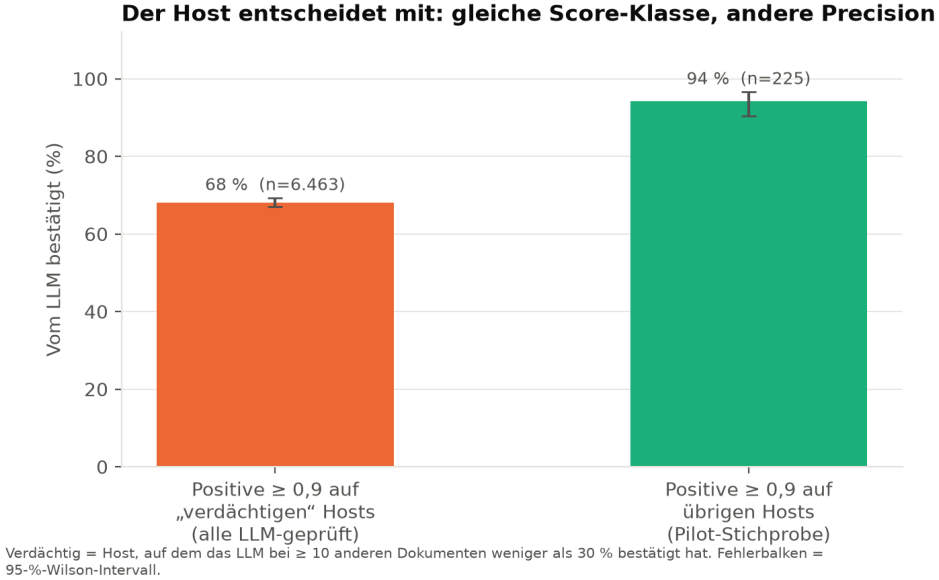


Abbildung 7.5: Precision (LLM-Bestätigungsrate) bei Score ≥ 0,9 auf verdächtigen und übrigen Hosts.

Manuelle Gegenprobe. Als einzige Messung gegen eine menschliche Bewertung wurden 20 Dokumente von Hand geprüft: 17 von 20 stimmten mit dem LLM überein (85 %). Die ungeprüften

TF-IDF-Positive mit Score $\geq 0{,}9$ waren 8 von 8 echt, von den LLM-bestätigten 5 von 6 richtig, von den LLM-verworfenen 4 von 6 richtig. Die beiden fälschlich verworfenen Dokumente waren laut LLM-Begründung „Studienordnung“, vermutlich weil das Modell nur die ersten 8.000 Zeichen sieht und einige Handbücher mit einer Ordnung beginnen (Vermutung, nicht geprüft). Mit $n = 20$ ist die Aussage grob (Übereinstimmung etwa 64–95 %); weitere Prüfungen wurden nicht durchgeführt. Das LLM macht damit Fehler in beide Richtungen, die Zahl verpasster Handbücher ist höher, als die Tabellen zeigen.

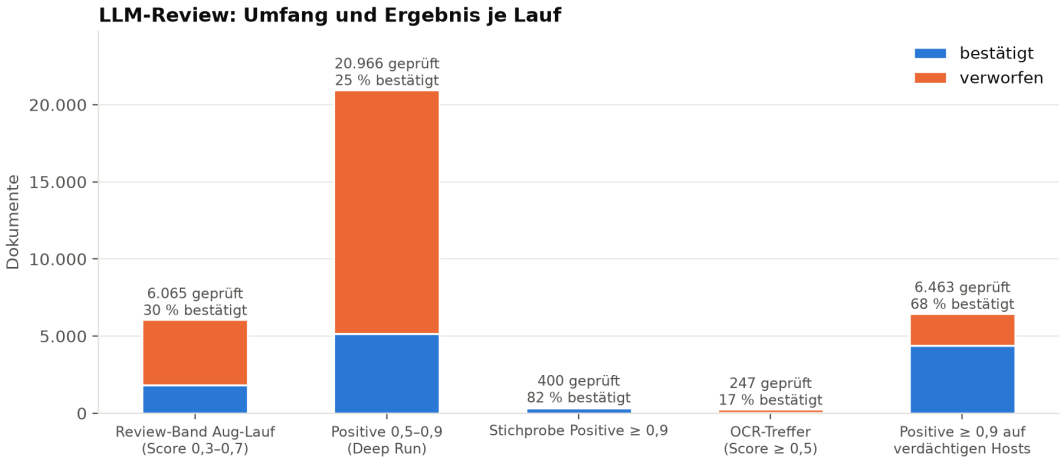


Abbildung 7.6: LLM-Läufe: Umfang und Bestätigungsrate.

Ein Fehler in der Auswertung. Ein Fehler in der Evaluation (`rescore_decisions`) überschrieb jedes LLM-Urteil beim Neubewerten mit der Score-Schwelle: Ein vom LLM abgelehnter Fehlalarm kippte zurück auf „positiv“, sobald sein Score $\geq 0{,}5$ war. Damit waren *alle* bisherigen LLM-Ergebnisse in der Auswertung stillschweigend wirkungslos. Der Fix: Datensätze mit `llm_reviewed=True` werden nie neu bewertet, das LLM-Urteil ist Wahrheit. Wirkung: positive Dokumente 14.133 → 13.565, Review 5.852 → 2.122, Abdeckung 314 → 303. Die *ehrliche* Zahl war also kleiner als die zuvor berichtete.

7.6 OCR für gescannte PDFs

Rund 3.600 PDFs lieferten keinen Text (`empty_document`), landeten im Review-Band und sollten nie stillschweigend im Negativ verschwinden. Ein OCR-Fallback (PyMuPDF rastert, Tesseract mit `deu+eng`) wurde gebaut und lokal auf 3.553 Scans angewendet: 3.118 lieferten Text, nur 247 erreichten einen Score $\geq 0{,}5$ und gingen ans LLM, 42 wurden bestätigt (Abb. 7.7). Der Ertrag (rund 1 % der Ausgangsmenge) steht in keinem Verhältnis zum Aufwand (lokale Tesseract-Installation, mehrere parallel laufende Prozesse über viele Stunden); ein klassisches Beispiel für einen Ansatz, der gebaut wurde, weil er plausibel war, und dessen Grenznutzen erst die Messung zeigte.

7.7 Erwogene Alternativen

- **Regex auf „Modulhandbuch“ im Dateinamen/Text:** scheitert in beide Richtungen (Handbücher ohne das Stichwort im Namen, Verordnungen und Formulare, die „Modulhandbuch“ erwähnen).

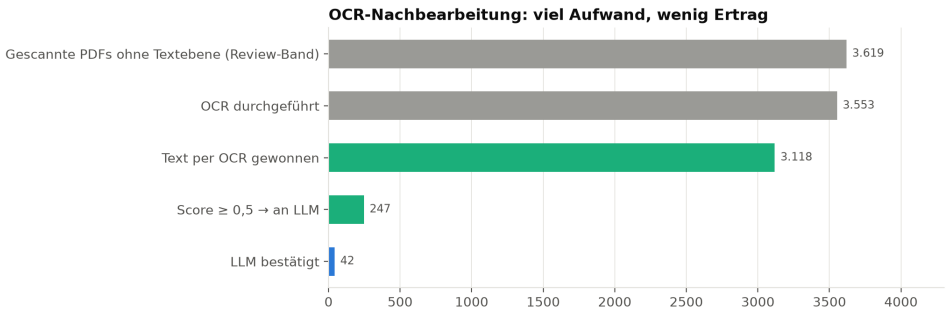


Abbildung 7.7: OCR-Trichter: von den Scans ohne Textebene bis zu den bestätigten Handbüchern.

- **Transformer-Finetuning (z. B. BERT):** bei 202 bzw. 356 Dokumenten ungeeignet; bei 2.695 Dokumenten denkbar, aber die LLM-Zweitstufe liefert dasselbe ohne Training.
- **LLM für alle Dokumente:** bei 164.396 PDFs und rund 2.300 Eingabetoken je Dokument wären das gut 375 Millionen Token; mit dem TF-IDF-Filter genügen rund 34.141 Dokumente (Kosten 107 $ statt etwa 500 $). Die zweistufige Architektur spart knapp 80 % der LLM-Aufrufe.
- **Rein manuelles Prüfen:** Bei 30 s je Dokument wären das rund 285 Stunden für die LLM-geprüften Dokumente allein (Abb. 8.16).

8 Ergebnisse und Kennzahlen

Dieses Kapitel beantwortet die Leitfragen aus Kapitel 1 mit Zahlen. Alle Werte stammen aus der Abschlussauswertung und den S3-Auflistungen; Stand ist der endgültige Datenstand vom 6. Oktober 2026.

8.1 Kennzahlen auf einen Blick

Tabelle 8.1: Zentrale Kennzahlen des Projekts.

Kennzahl	**Wert**
Hochschulen in der Hochschulliste	404
Heruntergeladene PDFs (URL-eindeutig, alle Läufe)	164.396
Objekte / Datenmenge in S3 (inkl. physischer Duplikate)	193.438 / 154,1 GB
Laufzeit Großlauf August / Deep Run	6,3 h / 12,0 h
Durchsatz August / Deep Run (PDFs pro Stunde im Mittel)	10.467 / 11.016
TF-IDF-Positive vor jeder Prüfung (Schwelle 0,5)	44.437
Vom LLM geprüfte Dokumente	34.141
davon bestätigt / verworfen	11.771 / 22.367
Endliste streng (nur LLM-bestätigt)	11.771
Endliste erweitert (zusätzlich TF-IDF $\geq$ 0,9, ungeprüft)	19.049
Geschätzt echte Modulhandbücher in der erweiterten Liste	18.628 (18.348–18.801)
Eindeutig nach Host und Dateiname (streng / erweitert)	10.165 / 16.316
Hochschulen mit $\geq$ 1 Handbuch (streng / erweitert)	255 (63 %) / 263 (65 %)
Hochschulen ohne jedes Dokument bzw. ohne Fund	39
Median Handbücher je Hochschule (erweitert)	4
Anteil der zehn ergiebigsten Hochschulen am Ergebnis	40 %

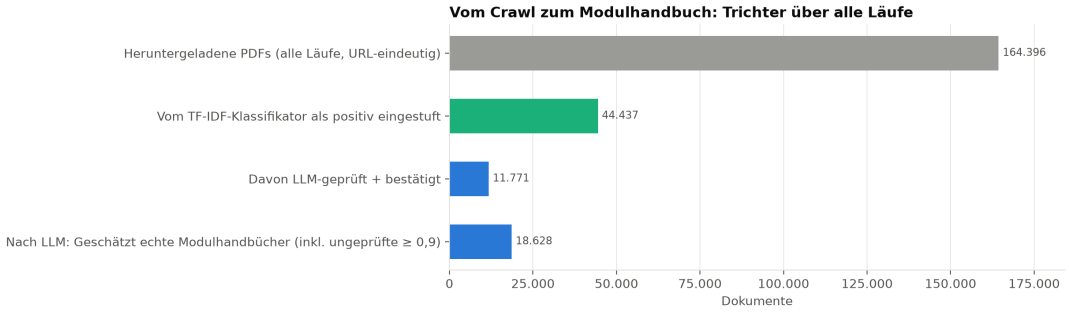


Abbildung 8.1: Trichter vom Crawl (164.396 PDFs) über die TF-IDF-Positive und die LLM-Prüfung bis zu den geschätzt echten Handbüchern.

8.2 Wie lange dauert ein Scraping-Lauf?

Abbildung 8.2 und Tabelle 5.2 zeigen die Läufe. Der Großlauf im August dauerte 6,3 h für 65.944 PDFs (rund 10.467 PDFs pro Stunde, im Spitzenbereich bis rund 11.900 pro Stunde), der Deep Run 12,0 h für 132.193 PDFs (rund 11.016 PDFs pro Stunde, Spitze rund 14.400). Der Seeds-only-Lauf holte 431 PDFs von 134 Hosts in unter vier Minuten. Der Median pro Hochschule lag im August bei 285 Sekunden. Entscheidend ist, dass die Läufe von wenigen großen Hochschulen dominiert werden, die Mehrzahl der Hochschulen war in Minuten abgearbeitet.

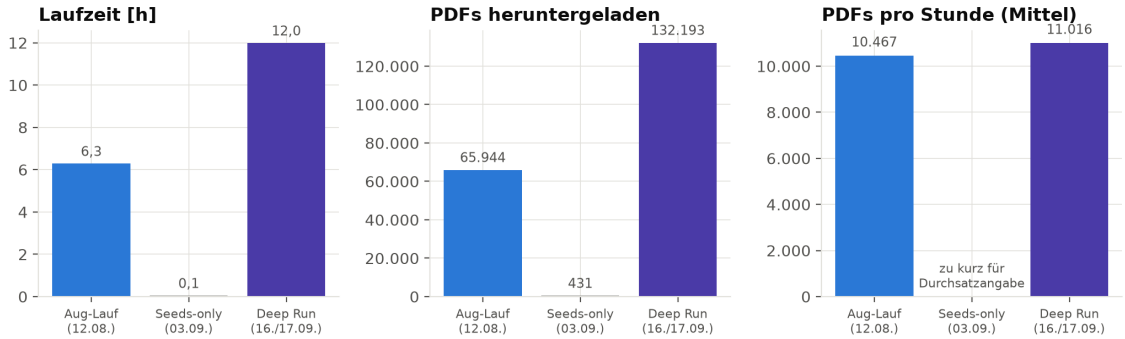


Abbildung 8.2: Laufzeit, Dokumentenzahl und Durchsatz der drei Läufe.

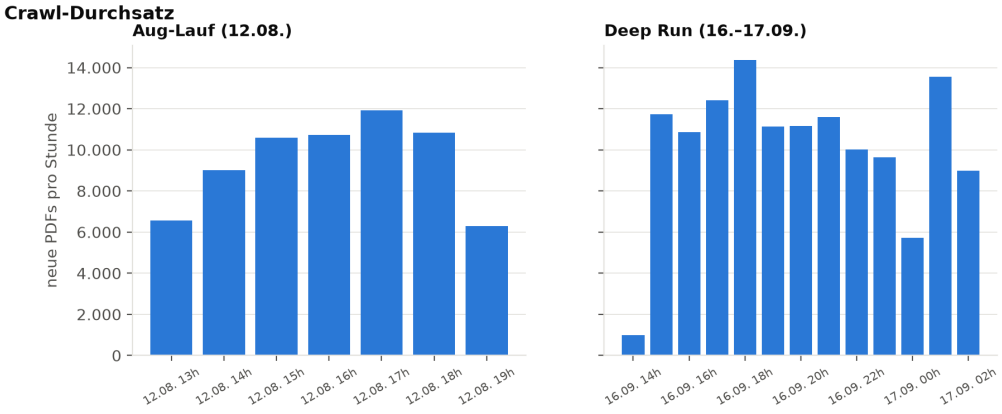


Abbildung 8.3: Durchsatz (neue PDFs pro Stunde) im Verlauf der beiden Großläufe.

8.3 Wie viele Dokumente werden gefunden, und was geschieht mit ihnen?

Abbildung 8.4 zeigt den Endstatus aller 164.396 Dokumente nach Lauf: LLM bestätigt (11.771), TF-IDF $\geq$ 0,9 ungeprüft (7.278), Positive aus anderen Läufen ungeprüft (6.393), LLM verworfen (22.367), TF-IDF negativ ungeprüft (114.926) und ungeklärt wegen fehlendem Text oder Fehlern (1.658). Die beiden Großläufe überlappen: 33.883 URLs kamen in beiden vor; im Deep Run gilt dann der jüngere Datensatz. Der Deep Run erreichte 718 Hosts, die der August-Lauf nicht kannte, der August-Lauf 296 Hosts, die der Deep Run nicht erreichte (Abb. 8.5).

8.4 Wie groß sind die gespeicherten Datenmengen?

Der S3-Bucket enthält zum Auswertungszeitpunkt 193.438 Objekte mit zusammen 154,1 GB (143,5 GiB); davon entfallen rund 55 GB auf den August-Lauf und rund 99 GB auf den Deep Run,

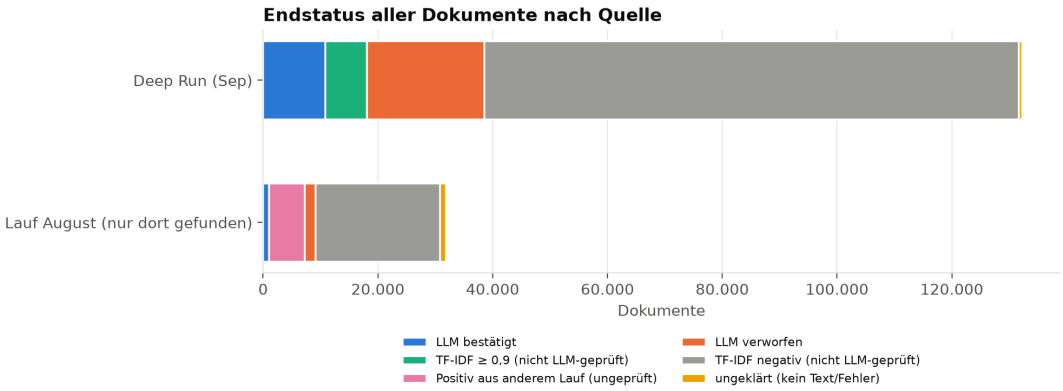


Abbildung 8.4: Endstatus aller Dokumente nach Lauf.

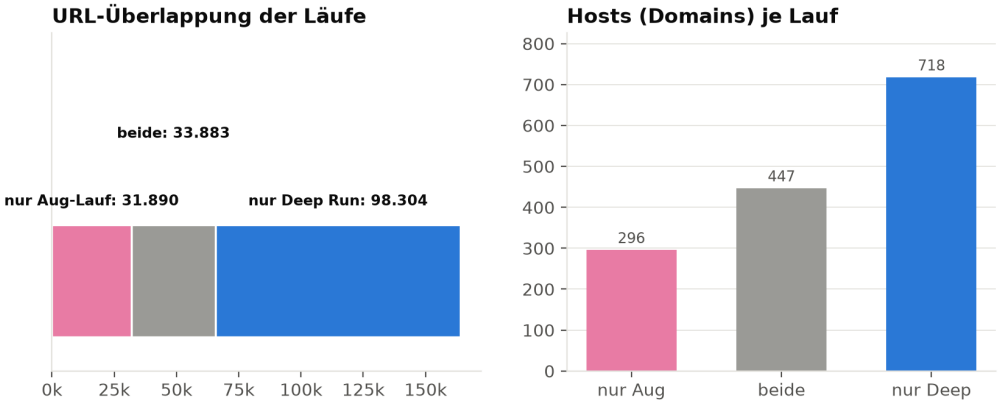


Abbildung 8.5: Vergleich von August-Lauf und Deep Run (URL-Überlappung).

die Seeds-Läufe wiegen unter 1 GB (Abb. 8.6). Da erzwungene Wiederholungsläufe dieselben PDFs unter neuer Job-ID ablegen, sind rund 15 % des Volumens physische Duplikate; die URL-eindeutigen Dokumente belegen 131,0 GB. Ein durchschnittliches PDF ist 0,80 MB groß, der Median liegt bei 0,24 MB; echte Modulhandbücher sind mit einem Median von 0,49 MB größer (typischerweise 100 KB bis 10 MB). Die gelisteten Modulhandbücher (erweiterte Liste) belegen zusammen rund 20,0 GB, die streng bestätigten rund 11,7 GB. Die Manifeste (Metadaten, Scores) wiegen insgesamt nur rund 0,2 GB.

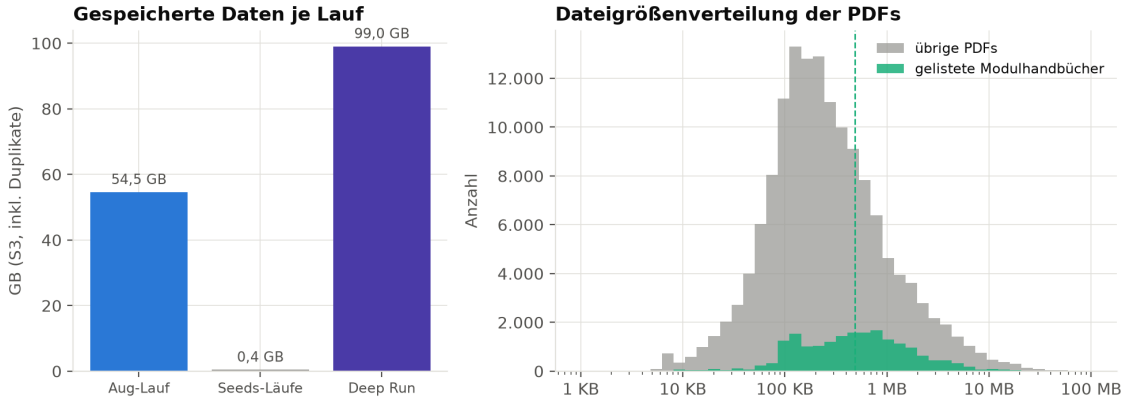


Abbildung 8.6: Gespeicherte Daten je Lauf und Dateigrößenverteilung der PDFs (logarithmische Achse).

8.5 Wie viele Hochschulen und Studiengänge werden erfasst?

Von 404 Hochschulen haben 255 (63 %) mindestens ein LLM-bestätigtes Handbuch; mit den ungeprüften Treffern oberhalb von 0,9 sind es 263 (65 %). Bei 158 Hochschulen liegen mindestens zehn Handbücher vor, bei 39 Hochschulen wurde *nichts* gefunden (kein Dokument, keine Seeds). Die Verteilung ist stark schief (Abb. 8.7): Der Median liegt bei 4 Handbüchern je Hochschule, die zehn ergiebigsten Hochschulen liefern 40 % aller Treffer. Ein Host allein (der Modulhandbuch-Server der Universität Augsburg) stellt über 1.800 Dokumente (Abb. 8.8).

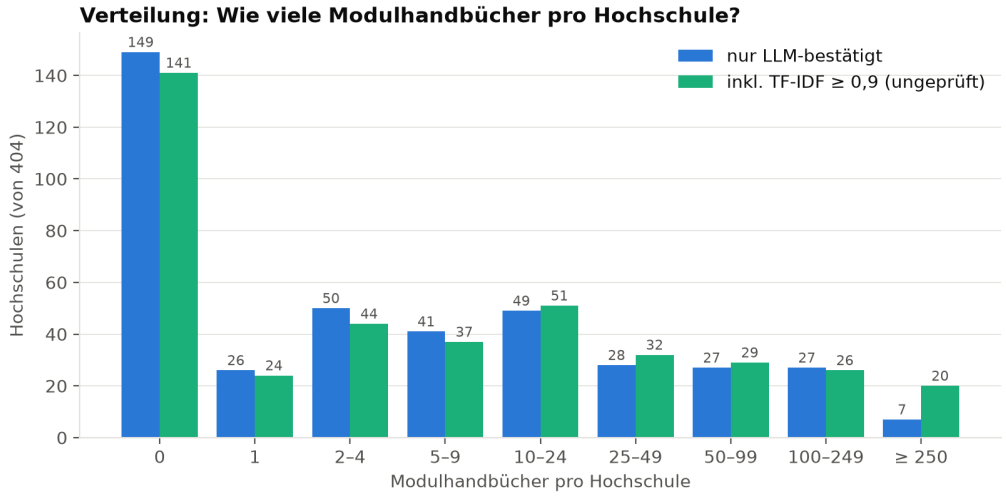


Abbildung 8.7: Verteilung der Treffer pro Hochschule.

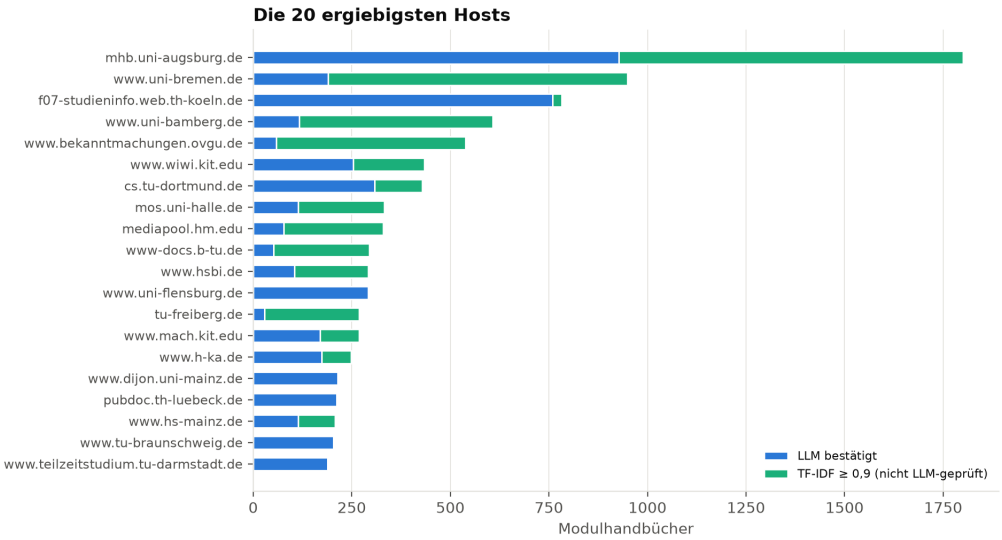


Abbildung 8.8: Die 20 ergiebigsten Hosts.

Studiengänge. Ein Modulhandbuch beschreibt in der Regel einen Studiengang (bzw. eine Prüfungsordnungsversion); die Zahl der Handbücher ist daher eine Näherung an die Zahl der erfassten Studiengänge, aber *keine* exakte Zählung: Es gibt Sammelhandbücher für mehrere Studiengänge, mehrere Versionen desselben Studiengangs und Handbücher für Teilmodule. Eine saubere Zählung der Studiengänge wäre erst mit einer inhaltlichen Extraktion möglich (Ausblick, Kapitel 10).

8.6 Wie unterscheiden sich kleine und große, öffentliche und private Hochschulen?

Die Abdeckung hängt stark von Typ, Träger und Größe der Hochschule ab (Abb. 8.9 bis 8.11):

- **Typ:** Universitäten 76 %, Künstlerische Hochschulen 70 %, Fachhochschulen/HAW 60 %, Verwaltungshochschulen 44 %.
- **Träger:** öffentlich-rechtlich 78 %, kirchlich 51 %, privat 39 %.
- **Größe:** Hochschulen unter 1.000 Studierenden liefern im Mittel nur rund 6,0 Handbücher und ca. 147 heruntergeladene PDFs; Hochschulen mit 15.000–30.000 Studierenden im Mittel rund 222 Handbücher bei rund 909 PDFs (Abb. 8.12).

Der Zusammenhang ist plausibel: Kleine private Hochschulen veröffentlichen Handbücher seltener oder in Login-Bereichen, haben kaum PDFs, nutzen JavaScript-Portale oder gar keine öffentlichen Handbücher. Große öffentliche Hochschulen haben oft Modul-Datenbanken mit tausenden Einzeldokumenten. Das heißt auch: Der Aufwand pro gefundenem Handbuch ist bei kleinen Hochschulen höher, der Nutzen je Crawl-Minute geringer. Grafik 8.13 zeigt die Streuung (Studierendenzahl gegen Treffer).

8.7 Wie hoch ist die Erkennungs- und Klassifikationsquote?

Die Antwort hängt davon ab, was als Referenz gilt (vgl. Kapitel 7):

- **Trainingsmetrik** (gruppierte Kreuzvalidierung): Precision 96,5 %, Recall rund 95 %.

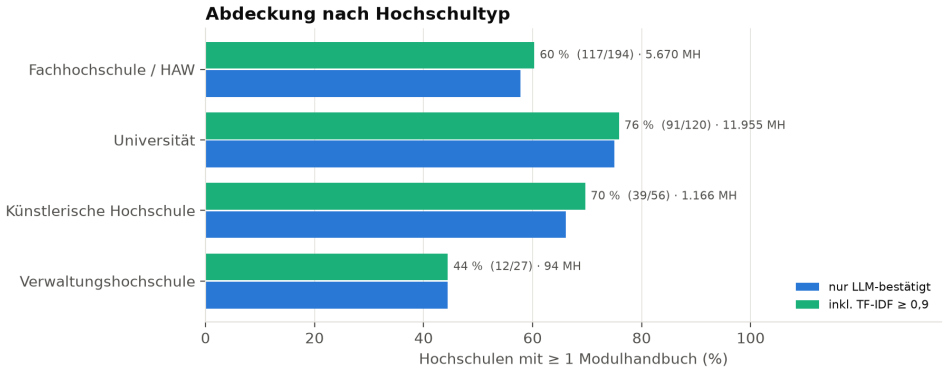


Abbildung 8.9: Abdeckung nach Hochschultyp.

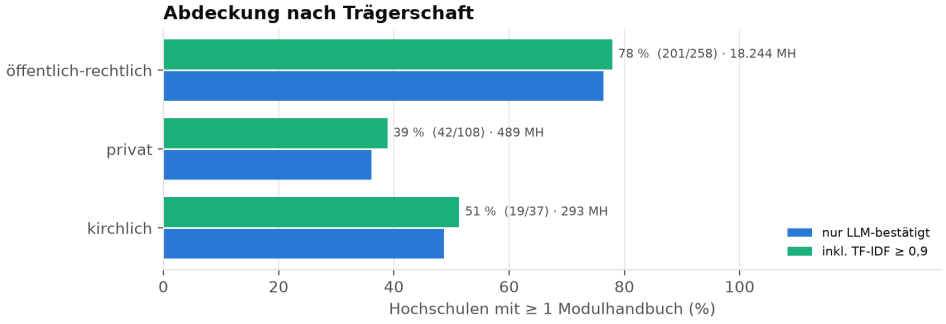


Abbildung 8.10: Abdeckung nach Trägerschaft.

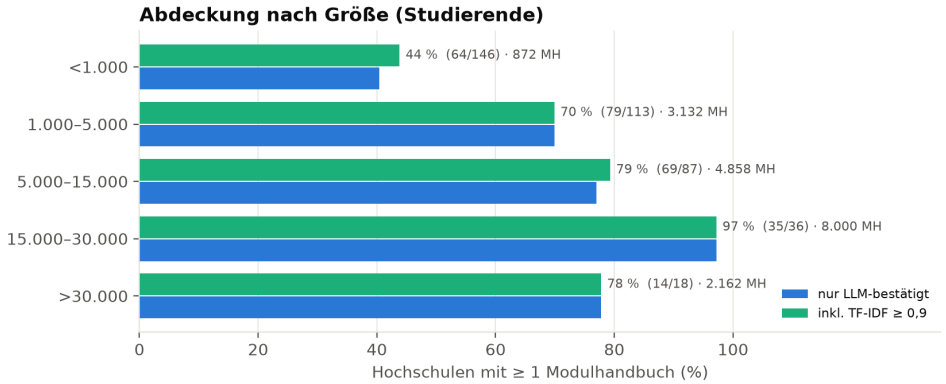


Abbildung 8.11: Abdeckung nach Hochschulgröße (Studierendenzahl).

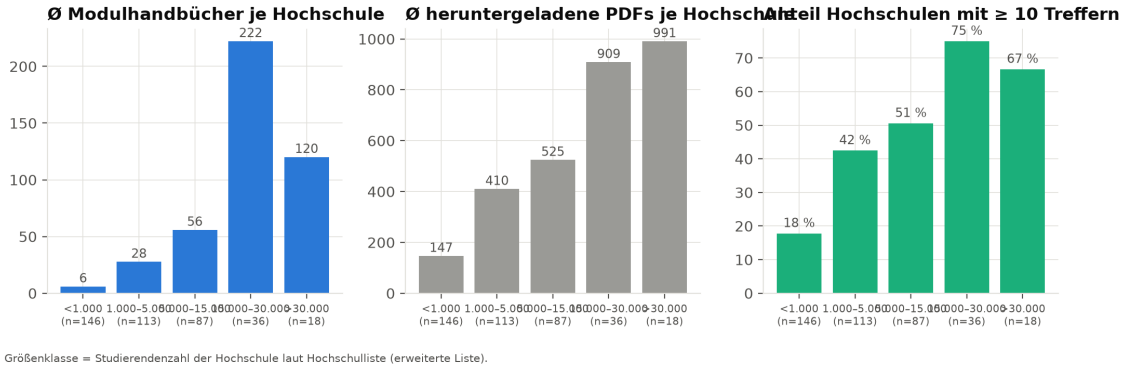


Abbildung 8.12: Mittlere Treffer, mittlere PDFs und Anteil ergiebiger Hochschulen nach Größenklasse.

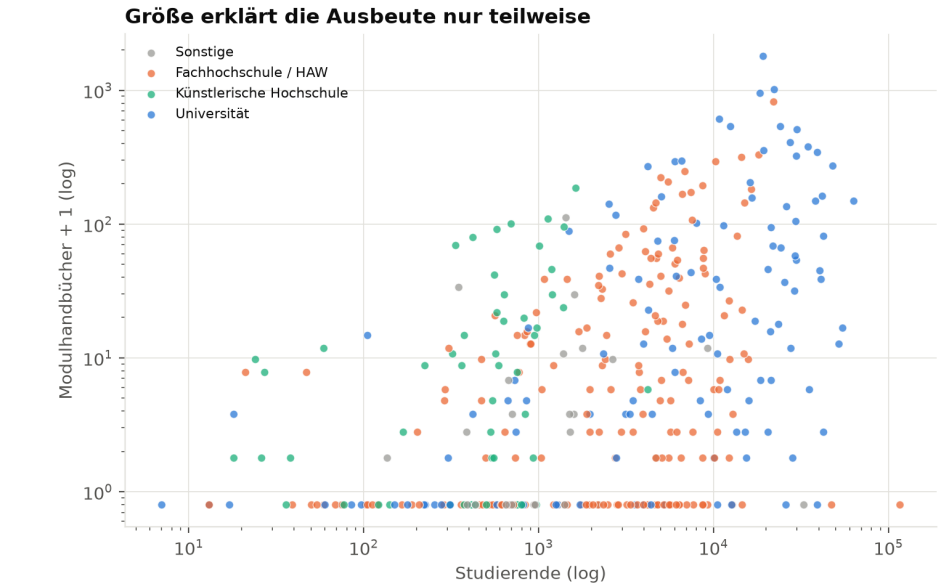


Abbildung 8.13: Studierendenzahl gegen Treffer pro Hochschule.

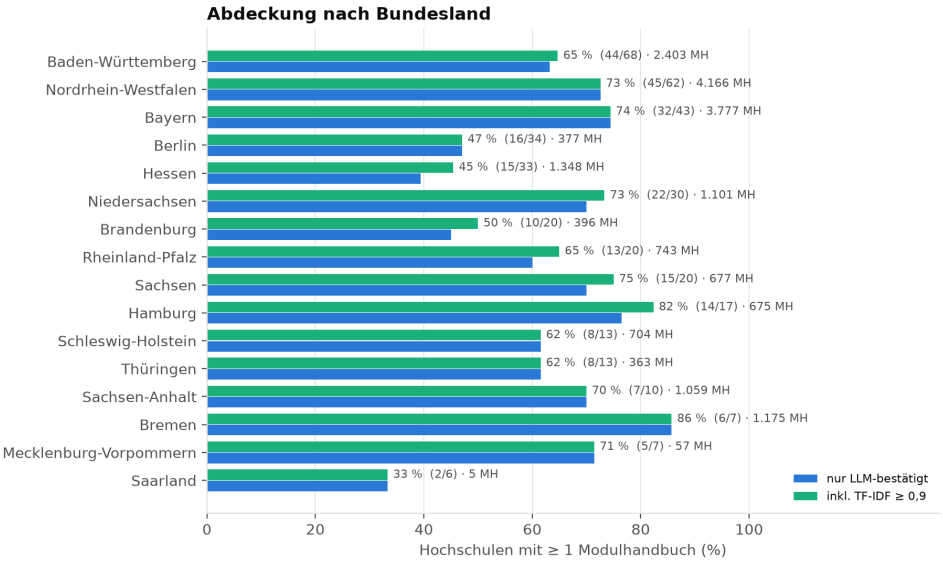


Abbildung 8.14: Abdeckung nach Bundesland.

- **Realer Crawl, Score 0,5–0,9:** Precision rund 25 % (Referenz LLM).
- **Realer Crawl, Score ≥ 0,9:** Precision 82,5 % (Pilot, $n = 400$).
- **Endliste erweitert:** geschätzt rund 18.628 echte von 19.049 gelisteten Dokumenten, das entspricht einer Precision von rund 98 % (gemessen am LLM-Urteil; Grundlage: 212 von 225 bestätigt in der Stichprobe auf den übrigen Hosts); die streng bestätigte Liste ist per Definition vom LLM bestätigt; ihre tatsächliche Fehlerquote entspricht der Fehlerquote des LLM, die nur die kleine manuelle Gegenprobe (17 von 20 Übereinstimmungen, 85 %) grob andeutet; die Referenz ist das LLM, nicht ein Mensch.
- **Recall:** unbekannt, nur grob abschätzbar: Gegenüber dem Erwartungswert von rund 40.000 Handbüchern liegen die 16.316 eindeutigen gelisteten Dokumente bei rund 41 % (eine Größenordnung, keine Messung). Aus den nicht LLM-geprüften Negativen mit Score 0,3–0,5 (18.252 Dokumente) lässt sich eine kleine Stichprobe von rund 383 verpassten Handbüchern schätzen, *sehr unsicher* (Abb. 8.15).

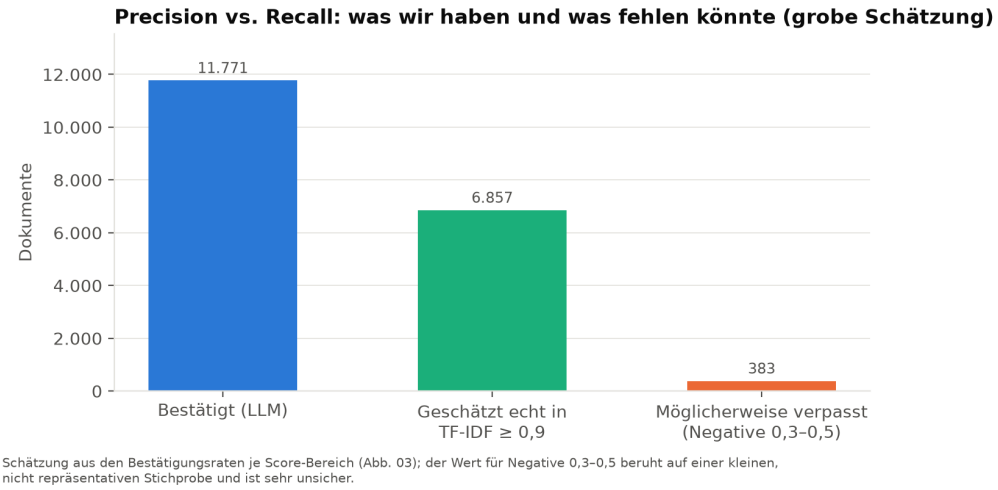


Abbildung 8.15: Was vorliegt und was fehlen könnte (grobe Schätzung).

8.8 Wie viele Dokumente müssen manuell nachbearbeitet werden?

Ohne die LLM-Zweitstufe müssten alle Dokumente der Grauzone manuell geprüft werden. Mit ihr bleiben:

- 1.658 Dokumente, bei denen weder TF-IDF noch OCR/LLM ein Urteil liefern konnten (kein Text, Fehler);
- 7.278 Positive mit Score ≥ 0,9 auf den übrigen Hosts, die nicht LLM-geprüft sind (geschätzt rund 94,2 % echt) und in der erweiterten, nicht aber in der strengen Liste stehen;
- 6.393 Positive aus dem August- und den Seeds-Läufen, die ungeprüft und in keiner der beiden Listen enthalten sind;
- die manuelle Gegenprobe von 20 Dokumenten (Abschnitt 7.5); weitere manuelle Prüfungen wurden nicht durchgeführt.

Bei einer Annahme von 30 Sekunden je Dokument ersparte die LLM-Zweitstufe rund 285 Stunden manueller Prüfung (Abb. 8.16).

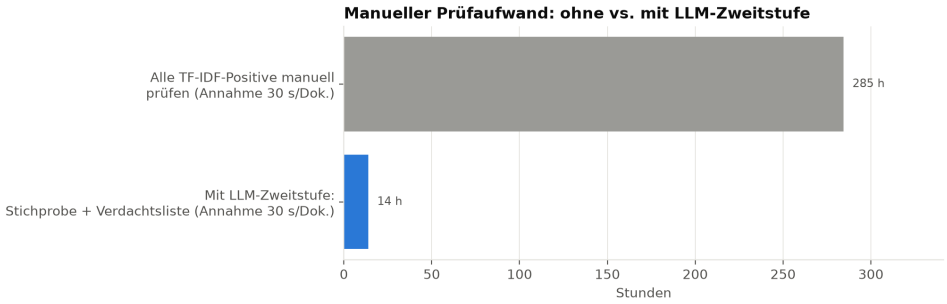


Abbildung 8.16: Manueller Prüfaufwand ohne und mit LLM-Zweitstufe (30 s je Dokument).

8.9 Wie viele Duplikate gibt es?

In der erweiterten Liste sind 2.733 Einträge Duplikate (gleicher Host und gleicher Dateiname), in der strengen Liste 1.606 (Abb. 8.17). Eindeutig bleiben 16.316 (erweitert) bzw. 10.165 (streng) Dokumente. Die Duplikate entstehen, weil dieselbe Datei über mehrere Seiten verlinkt ist oder in mehreren Läufen vorkommt; auf Speicherebene kommen die physischen Duplikate erzwungener Läufe hinzu (siehe oben). Die Deduplikation per Host und Dateiname ist eine Näherung; zwei Hochschulen mit gleichnamigen Dateien auf getrennten Hosts bleiben richtig getrennt.

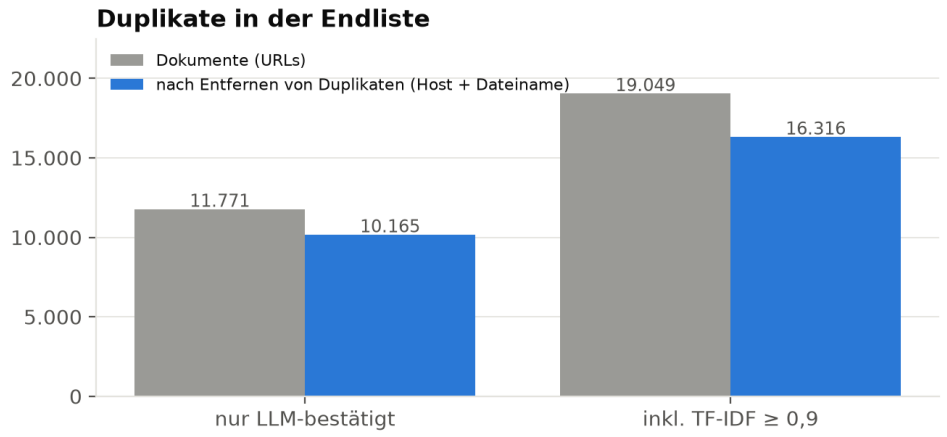


Abbildung 8.17: Duplikate in der Endliste.

8.10 Wie weit reichen die ältesten Dokumente zurück?

Das Erscheinungsjahr ist nur indirekt bestimmbar. Bei 10.543 der gelisteten Dokumente steht eine Jahreszahl im Dateinamen. Sie reicht von 2001 (nur 2 Dokumente) bis 2027 (Handbücher für künftige Semester). Der Median ist 2021, 61 % der Dokumente tragen ein Jahr ab 2020, 2.215 die Jahre 2015 bis 2019 und 1.864 ein Jahr vor 2015 (Abb. 8.18). Der Crawl ist also zu den aktuellen Jahrgängen verzerrt, hat aber einen langen Schwanz älterer Versionen: Hochschulen lassen alte Handbuchversionen online. Diese Angabe ist ein Näherungswert: Eine Jahreszahl im Namen ist nicht notwendig das Veröffentlichungsjahr, und viele Dateien haben gar keine.

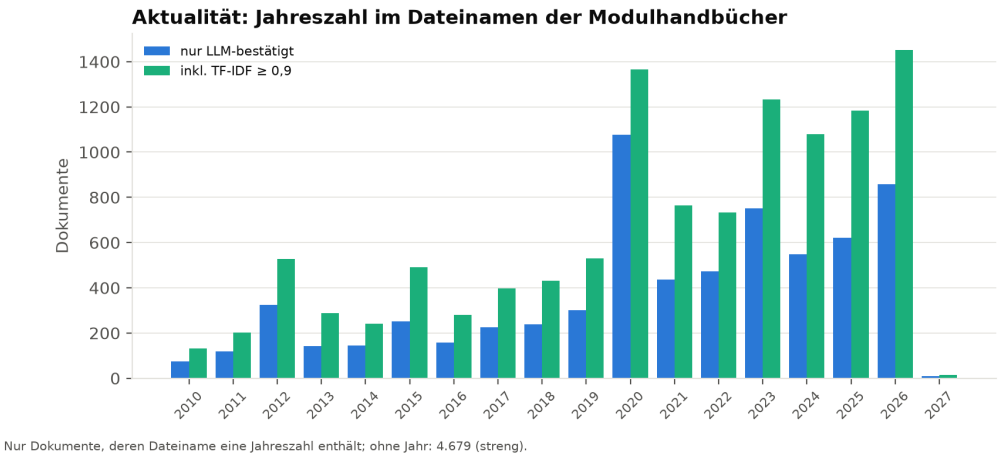


Abbildung 8.18: Jahreszahlen in den Dateinamen der gelisteten Dokumente.

8.11 In welcher Tiefe der Webseiten liegen die Dokumente?

Die Klicktiefe wurde im Manifest nicht protokolliert. Als Näherung dient die *Pfadtiefe der Fundseite* (Anzahl der URL-Segmente der Seite, die auf das Dokument verlinkt). Der häufigste Wert über alle PDFs ist 3, bei den gelisteten Handbüchern 5; 67 % der Handbücher liegen auf Fundseiten mit Pfadtiefe bis 4, Handbücher liegen also im Mittel *tiefer* als Durchschnitts-PDFs (Abb. 8.19). Das stützt die Erfahrung aus Kapitel 6: Eine feste Tiefe von 2, wie sie der erste Crawl nutzte, verpasst den Großteil. Eine echte Klicktiefenstatistik wäre möglich, wenn die Tiefe künftig im Manifest mitgeschrieben würde (Scrapy-Meta `depth`).

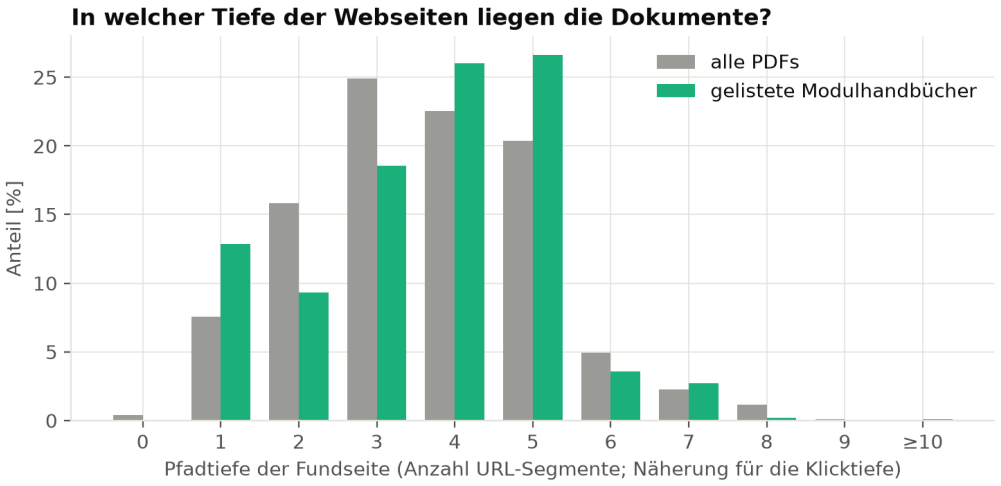


Abbildung 8.19: Pfadtiefe der Fundseiten (Näherung für die Klicktiefe) für alle PDFs und die gelisteten Modulhandbücher.

8.12 Welche Datenmenge wäre bei einer deutschlandweiten Vollerhebung zu erwarten?

Die Hochschulliste umfasst 404 Hochschulen und damit praktisch das gesamte Hochschulwesen; die *Breite* der Erhebung ist also bereits deutschlandweit, die *Tiefe* noch nicht. Für den Erwartungswert von rund 40.000 Modulhandbüchern (Fachschätzung) liefert eine lineare Hochrechnung

bei gleicher Ausbeute (8,8 PDFs je echtem Modulhandbuch) rund 353.008 zu sammelnde PDFs, rund 281 GB Speicher und rund 73.311 LLM-geprüfte Dokumente mit Kosten von etwa 230 $ (Abb. 8.20). Die Rechenkosten des Crawls bleiben auch dann einstellig. Die Hochrechnung ist eine *Untergrenze*: Die noch fehlenden Handbücher liegen auf den schwer erreichbaren Seiten (kleine, private, JavaScript, Login), die Ausbeute pro PDF wird dort schlechter sein. Wesentlich realistischer als mehr Crawl-Volumen ist daher ein anderer Weg zu den fehlenden Handbüchern (Discovery, Seitenebene, manuelle Nachsuche in der Liste der 39 leeren Hochschulen und der rund 110 Hochschulen mit Dokumenten, aber ohne bestätigtes Handbuch).

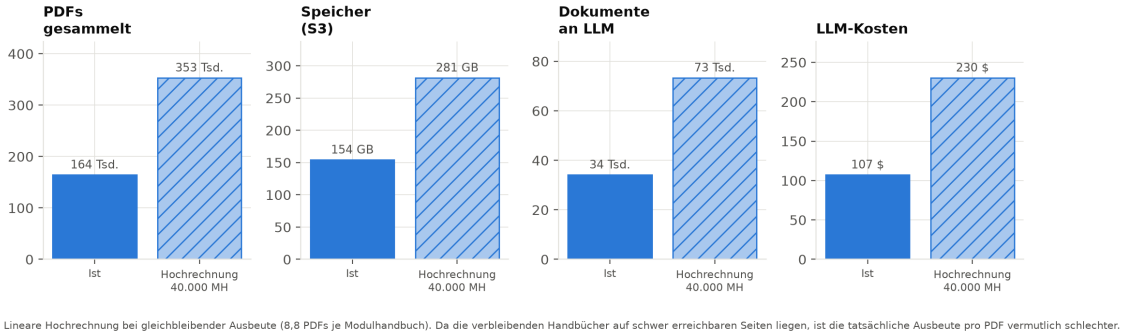


Abbildung 8.20: Lineare Hochrechnung der Erhebung für 40.000 Modulhandbücher (Ist: durchgezogen, Hochrechnung: schraffiert).

9 Diskussion: Sackgassen, Lehren und Grenzen

Das Projekt hat mehr Wege verworfen, als es am Ende beschritten hat. Dieses Kapitel sammelt sie bewusst an einer Stelle, ordnet sie ein und zieht Schlüsse. Es folgt eine ehrliche Bewertung der Aussagekraft der Ergebnisse.

9.1 Wege, die nicht (oder nur teilweise) funktioniert haben

Tabelle 9.1 listet die Sackgassen und teilweisen Fehlschläge in der Reihenfolge ihres Auftretens. Der *Nutzen* der Sackgassen liegt in dem, was sie gezeigt haben: oft die Ursache des eigentlichen Problems.

9.2 Lehren

1. **Der Engpass wandert.** Zuerst war es die Plattform, dann die Laufzeit, dann der Recall, am Ende die Precision. Jede Verbesserung deckt das nächste Problem auf. Eine frühe Messung am Ziel (statt auf dem Papier) hätte die Reihenfolge verkürzt.
2. **Trainingsmetriken sind keine Betriebsmetriken.** 96,5 % Precision in der gruppierten Kreuzvalidierung, 25 % im Crawl. Der Unterschied liegt in der Verteilung der Negativbeispiele, nicht im Modell. Wichtigste Lehre für ML-Projekte: eine Stichprobe aus der *echten* Verteilung früh labeln.
3. **Eine zweite, stärkere Stufe lohnt für die Grauzone.** Ein billiges Modell filtert, ein teures prüft nur die Unsicheren. Das spart knapp 80 % der LLM-Aufrufe gegenüber einer Vollprüfung.
4. **Die Evaluation braucht eigene Tests.** Der Fehler, der LLM-Urteile überschrieb, war nicht im Modell, sondern in der Auswertung, und er verfälschte alle Zahlen zuvor.
5. **Erst messen, dann bauen (und danach wieder messen).** OCR und Firecrawl waren plausibel, brachten aber kaum Ertrag; Seeds-only brachte *in Minuten* +43 Hochschulen.
6. **Infrastruktur wegwerfbar, Daten nicht.** Der Sandbox-Verlust war verkraftbar, weil der Aufbau skriptbar war; verloren schmerzte, was nur in S3 lag.
7. **Die Qualität hängt am Host, nicht nur am Score.** Bei Score $\geq$ 0,9 waren die Dokumente auf den 197 verdächtigen Hosts nur zu 68 % echt, auf den übrigen zu 94,2 %. Eine gezielte Nachprüfung der schlechten Hosts brachte mehr als eine breite Stichprobe.
8. **Randbedingungen sind Anforderungen.** SCPs, kurzlebige Schlüssel und Proxy waren keine Störungen, sondern gaben die Architektur der Nachbereitung vor.

Tabelle 9.1: Verworfene oder gescheiterte Wege und ihre Lehre.

Nr.	Weg	Befund	Lehre / Ersatz
1	Lambda für alle Läufe, Tiefe/Seiten hochdrehen	15-min-Limit, RAM-Pufferung; Timeouts bei Tiefe ≥ 2 mit vielen Seiten	Fargate-Bulk-Pfad; Lambda nur noch interaktiv
2	Synchrone Webapp-Antwort	HTTP-Timeout bei längeren Läufen; Endlosschleife bei Mehrfach-Formaten	Asynchrone Jobs mit Polling; Visited-Store; harte Obergrenzen
3	Terraform als Infrastruktur-Wahrheit	Zustand und Handanlagen liefen auseinander; Werkzeuge im Dev-Setup nicht nutzbar	AWS-CLI + JSON-Vorlagen, später Bootstrap-Skript
4	Ungesteuerter Tiefen-Crawl	TU Dortmund: 4 echte von 934 Dokumenten	Best-First, Sitemap, Subdomain-Erkennung
5	Sitemaps als Quelle für tiefe Dateien	Konstanz hat keine Sitemap; tiefe Dateien stehen nirgends	Suchindizes (Common Crawl, Serper)
6	Google Custom Search	Web-weite Suche entfernt (50-Domain-Grenze)	Serper (SERP-API)
7	DuckDuckGo-Scraping	CAPTCHA aus der Cloud	verworfen
8	Firecrawl-Rendering (JS)	Für IU keine PDFs; Pipeline gebaut, nicht im Großlauf getestet	offen; Playwright-basiertes Rendering als Ausblick
9	Schwelle 0,5 für maximalen Recall	Positiv-Flut, Precision 25–60 % in der Praxis	LLM-Zweitstufe; Schwelle 0,8–0,9 empfohlen
10	OCR für alle Scans	3.553 Scans, nur 42 bestätigte Handbücher (rund 1 %)	Aufwand/Ertrag schlecht; nur für Review-Band sinnvoll
11	Re-Scoring in der Evaluation überschrieb LLM-Urteile	Falsche, zu optimistische Abdeckung (314 statt 303)	LLM-Urteil hat Vorrang; Test-Disziplin
12	DynamoDB-Dedup auf Hochschulebene bei Discovery-Wiederholung	Discovery lief nie für bereits gecrawlte Hochschulen	Seeds-only-Modus
13	Mantle-Endpunkt von Bedrock	per SCP verboten (403)	nativer Bedrock-Client
14	Langlebiger Bedrock-Schlüssel	vom Account nicht erlaubt; Schlüssel verfallen nach 30 min bis 12 h	Chunk-Verarbeitung, wiederaufnehmbar
15	Serper bezahlt / `filetype:`	Free-Tier erlaubt weder `filetype:` noch mehr als 10 Treffer	Free-Tier-Anpassung; Recall begrenzt
16	SQS-Fan-out für Breite	Roadmap-Punkt, nie gebaut	durch Fargate ersetzt
17	Menschen-Review als Hauptmittel der Qualitätssicherung	Skala (zehntausende Kandidaten)	LLM-Zweitstufe, Mensch nur für Stichprobe

9.3 Grenzen und Bedrohungen der Validität

Das LLM ist die Referenz, nicht ein Mensch. Alle Precision-Angaben messen die Übereinstimmung mit Claude Haiku 4.5. Ein systematischer LLM-Fehler (z. B. zu strenge Auslegung von „Sammelhandbuch“) steckt in allen Zahlen. Die einzige Messung gegen eine menschliche Bewertung ist eine Gegenprobe von nur 20 Dokumenten (17 Übereinstimmungen, 85 %); das LLM sieht zudem nur die ersten 8.000 Zeichen und hat in der Gegenprobe zwei echte Handbücher als „Studienordnung“ verworfen. Es macht also Fehler in beide Richtungen.

Nicht alles ist LLM-geprüft. Die 7.278 Positive mit Score $\geq 0{,}9$ auf den übrigen Hosts sind ungeprüft; ihr Anteil echter Treffer (94,2 %) ist aus einer Stichprobe von 225 Dokumenten geschätzt und setzt Repräsentativität voraus. Die 6.393 Positive aus dem August- und den Seeds-Läufen sind ungeprüft und in keiner Liste enthalten. Die Nachprüfung der 6.463 Dokumente auf den 197 verdächtigen Hosts hat die Precision der Endliste deutlich verbessert (von etwa 89 % auf etwa 98 %); weitere Prüfungen wurden nicht durchgeführt, die Ergebnisse gelten so, wie sie sind.

Der Recall ist nur grob bekannt. Der Erwartungswert von rund 40.000 ist eine Fachschätzung, keine Messung. Es gibt keine unabhängig gezählte Grundgesamtheit pro Hochschule. Die Schätzung verpasster Handbücher unter den Negativen stammt aus einer kleinen, nicht repräsentativen Stichprobe.

Definition. Ob Sammelhandbücher, Modulkataloge oder Studienverlaufspläne zählen, ist Auslegung. Die strenge Definition (21 % der Fehlalarme sind Einzelblätter) ist eine Entscheidung zugunsten der Precision; ein breiterer Begriff würde die Ergebnisliste vergrößern.

Verzerrungen. (1) Das Trainingsset ist verzerrt zu Hochschulen mit großem Online-Angebot. (2) Die Hochschulliste enthält mehr Fachhochschulen und private Hochschulen als Universitäten, die Abdeckung dort ist am schlechtesten. (3) Der Crawl ist zu aktuellen Jahrgängen verzerrt. (4) Die Klicktiefe ist eine Näherung (sie wurde im Manifest nicht protokolliert).

Ethik und Recht. Der Crawler beachtet robots.txt, drosselt Anfragen und greift ausschließlich öffentlich zugängliche Dokumente ab. Die Ausnahme für Discovery-Seeds (robots-Bypass für einzelne, per Suchindex bekannte Handbuch-URLs, Abschnitt 6.6.1) ist standardmäßig aus und wurde nur für Hochschulen mit pauschaler Sperre eingesetzt; sie ist eine bewusste Abwägung, keine Selbstverständlichkeit. Es werden keine personenbezogenen Daten gezielt erhoben. Die Urheberrechtslage bei der Weitergabe der gesammelten Dokumente wurde nicht geprüft; der Bericht ist kein juristisches Gutachten.

10 Fazit und Ausblick

10.1 Fazit

Das Projekt hat aus einem Scrapy-Prototyp eine skalierbare Erhebungs- und Klassifikationsplattform auf AWS gemacht und sie an einem realen, großen Anwendungsfall erprobt: 164.396 PDFs von 404 Hochschulen, zwei Großläufe mit zusammen rund 18 Stunden Laufzeit, 154,1 GB Daten, eine Endliste von 19.049 Dokumenten mit geschätzt 18.628 echten Modulhandbüchern und einer Abdeckung von 65 % der Hochschulen. Der Aufbau ist modular genug, dass ein anderer Dokumenttyp ein neues Profil und ein neues Modell braucht, nicht aber einen neuen Crawler. Das ist die Brücke zu KIWIT.

Tabelle 10.1 bewertet die Akzeptanzkriterien aus Kapitel 1 gegen das Ergebnis.

Tabelle 10.1: Akzeptanzkriterien und Ergebnis.

Kriterium	**Erfüllt?**	**Begründung**
A1 vollständig durchsuchen	weitgehend	Alle 404 Hochschulen wurden bearbeitet; Seiten mit JavaScript, Login oder anderer Domain nicht erreichbar.
A2 alle Handbücher finden (Recall)	**teilweise**	Rund 41 % des Erwartungswerts; 39 Hochschulen ohne Fund, rund 110 mit Dokumenten, aber ohne bestätigtes Handbuch (streng).
A3 Nicht-Handbücher erkennen (Precision)	ja, durch LLM-Stufe	TF-IDF allein reicht nicht (25–82 %); die streng bestätigte Liste ist vom LLM geprüft, die erweiterte Liste hat geschätzt rund 98 % Precision (gemessen am LLM).
A4 strukturiert speichern	ja	S3-Ablage mit Manifest (Score, Quelle, Lauf, LLM-Begründung); Transparenz je Entscheidung. Eine Datenbank für Inhalte ist offen.

Das wichtigste inhaltliche Ergebnis ist *negativ* und nützlich: Der Klassifikator, so gut er auf dem Papier war, genügte als alleiniger Filter nicht. Die Kombination aus billigem Massenfilter und LLM-Zweitstufe war die Antwort, und die Ehrlichkeit der Evaluation (gruppiert nach Hochschule, Pilotstichprobe, korrigierter Auswertungsfehler) hat dieses Ergebnis überhaupt erst sichtbar gemacht. Das wichtigste betriebliche Ergebnis: Eine befristete Sandbox verlangt, dass Aufbau und Daten wiederherstellbar sind.

10.2 Ausblick

10.2.1 Qualität und Recall

- **Schwelle und Modell.** Im Live-Crawl die vorgeschlagene Schwelle (0,8) statt 0,5 verwenden; den Dateinamen als Merkmal aufnehmen (Abschnitt 7); Dateiname und Host als Merkmale prüfen; mit den über 34.000 LLM-Labels als *Hard Negatives* nachtrainieren; mehr Text als nur die ersten 8.000 Zeichen an das LLM geben. Die dazu nötige Retrain-Strecke aus Manifesten (`ingest -manifest` mit S3-Abruf) ist vorhanden.

- **Manuelle Referenz.** Von der vorbereiteten Stichprobe von 120 Dokumenten wurden nur 20 geprüft; eine vollständige Auswertung würde die LLM-Genauigkeit belastbarer messen.
- **Ungeprüfte Positive.** Die 7.278 ungeprüften Positive (rund 18 $) und die 6.393 Positive aus dem August-Lauf (rund 16 $) ließen sich bei Bedarf durch das LLM schicken.
- **Seitenebene statt Dateiebene.** JavaScript-Portale (z. B. IU) liefern kein PDF; die Klassifikation von gerenderten Katalogseiten ist gebaut und wartet auf einen Einsatz mit Playwright oder Firecrawl.
- **Manuelle Nachsuche** für die 39 leeren und die kleinen privaten Hochschulen, deren Handbücher nicht online liegen oder hinter Login.

10.2.2 Inhaltliche Erschließung

Aus „ist ein Modulhandbuch“ soll „was steht darin“ werden: Module, ECTS-Punkte, Lernziele und Prüfungsformen als strukturierte, vergleichbare Daten. Für diese Extraktion ist ein LLM fachlich naheliegend; erst damit lassen sich Studiengänge zählen und systematisch vergleichen, wie es der ursprüngliche Anwendungsfall der FH Wedel verlangt.

10.2.3 Plattform und Betrieb

- Modell-Artefakte in S3 mit Registry versionieren, sodass Modellwechsel ohne Neubau des Images möglich ist.
- Scoring vom Crawl entkoppeln (S3-Ereignis lösen Klassifikations-Lambda aus) und Jobstatus in einer Tabelle halten (Roadmap Phase 3–4).
- Secrets Manager statt `.env`-Dateien, echtes TLS-Zertifikat statt selbstsigniertem, SQLite → RDS bei mehreren Schreibern, Auto-Scaling hinter einem Load Balancer.
- Automatisierte Tests und Monitoring (Fehlerraten, Review-Band-Anteil als Drift-Indikator).
- Webapp nur bei Bedarf betreiben: Die dauerhaft laufende Instanz kostete im Budget mehr als jeder Crawl; ein On-Demand-Betrieb wäre günstiger.

10.2.4 Anschluss an KIWIT

Für Forschungsprofil A der KIWIT-Gruppe (Makroebene) lassen sich damit künftig auch KI-bezogene Strategien, Richtlinien und Angebote erheben: Dazu genügen ein neues Extraktionsprofil mit Suchbegriffen, eine neue Spezifikation für die Zweitstufe und ein passendes Trainingsset. Die in diesem Projekt gewonnene Erfahrung, dass *Auffinden* und *Aussortieren* getrennte Probleme sind und beide eine eigene Lösung brauchen, sollte dabei von Beginn an berücksichtigt werden.

A Konfiguration der Läufe

Tabelle A.1: Konfigurationsgrenzen des Crawlers: Lambda/MVP gegenüber Fargate Deep Run.

Parameter	**MVP/Lambda**	**Deep Run**	**Bedeutung**
`CRAWL_MAX_DEPTH`	2	5	Link-Tiefe je Hochschule
`CRAWL_MAX_PAGES`	60	2.500	Seitenbudget (weich)
`CRAWL_MAX_PAGES_PER_SECTION`	–	60	Seitenkappe je Fakultäts-Sektion
`CLOSESPIDER_PAGECOUNT`	400	3.000	harte Seitengrenze
`CLOSESPIDER_TIMEOUT`	–	1.800 s	Zeitgrenze je Hochschule
`BULK_MAX_RUNTIME_SECONDS`	–	43.200	Wachhund (Gesamtlauf)
`CRAWL_MAX_SUBDOMAIN_SITEMAPS`	15	30	Sitemaps von Subdomains
`DOWNLOAD_MAXSIZE`	50 MB	50 MB	je Datei
`DOWNLOAD_TIMEOUT`	60 s	60 s	je Anfrage
`DOWNLOAD_DELAY`	1 s + AutoThrottle	dto.	je Domain
`CONCURRENT_REQUESTS`	4 (2 je Domain)	16	je Job
Rechenressource	Lambda 1.536 MB	Fargate 4 vCPU/16 GB	
`CLASSIFIER_UPPER_THRESHOLD`	0,65	0,50	obere Modellschwelle

B Metriken im Detail

Tabelle B.1: Metriken der Modellvarianten (gruppierte Kreuzvalidierung), jeweils am Recall-Ziel von 95 %.

Variante	PR-AUC	ROC-AUC	Recall	Prec.	F1	F2
MVP *content* (0,387)	0,957	0,941	0,955	0,871	0,911	0,937
MVP *content + Dateiname*	0,960	0,926	0,955	0,848	0,898	0,932
Sept. *content* (0,627)	0,988	0,973	0,950	0,965	0,958	0,953
Sept. *content + Metadaten* (0,509)	0,976	0,959	0,950	0,958	0,954	0,952

Konfusionsmatrizen. MVP *content*: 128 richtig positiv, 6 falsch negativ, 49 richtig negativ, 19 falsch positiv (202 Dokumente, 21 Hochschulen); bei Schwelle 0,5: Precision 0,907, Recall 0,873, 17 statt 6 übersehene Handbücher. September *content*: 1.873 / 98 / 656 / 68 (2.695 Dokumente, 269 Hochschulen); bei Schwelle 0,5: Precision 0,952, Recall 0,985 (1.942 TP, 29 FN, 625 TN, 99 FP).

C Reproduzierbarkeit

Quellcode und Commit-Historie: `https://github.com/JanSoetebeer/KML`. Die Auswertung lässt sich aus den Läufen neu erzeugen (aus dem Repo-Root):

```
python auswertung/build_final.py                    # Endliste aus allen Läufen
python auswertung/build_eval_manifests.py           # Manifeste je Variante
.venv\Scripts\python auswertung\make_charts.py      # Grafiken und Kennzahlen
```

Evaluationsberichte (`eval_*.html`) entstehen mit `python -m webscraper.evaluation` aus `webscraper/`.

D Weitere Auswertungsgrafiken

Die folgenden Grafiken der Abschlussauswertung werden im Text nicht direkt referenziert.

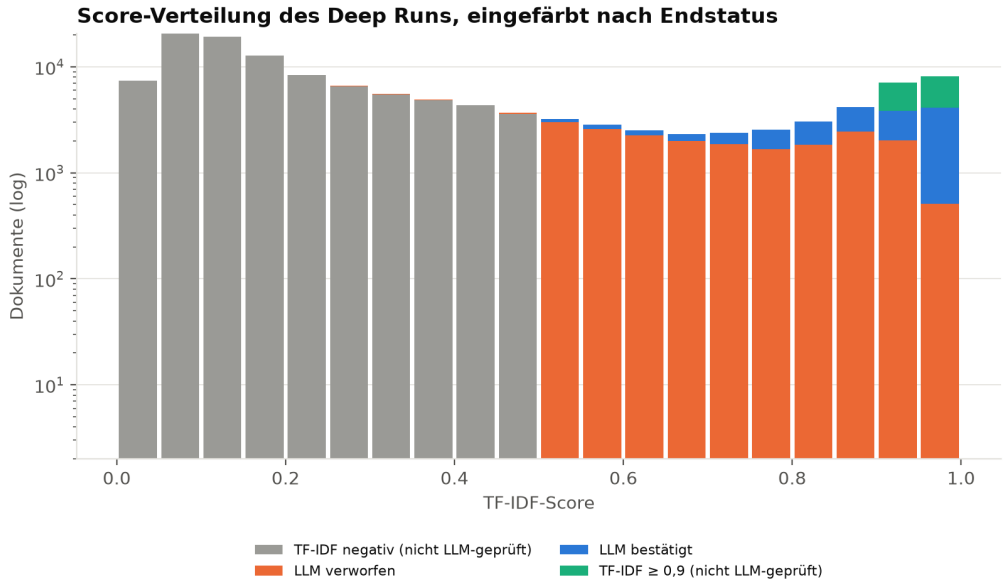


Abbildung D.1: Score-Verteilung nach Endstatus (logarithmische Achse).

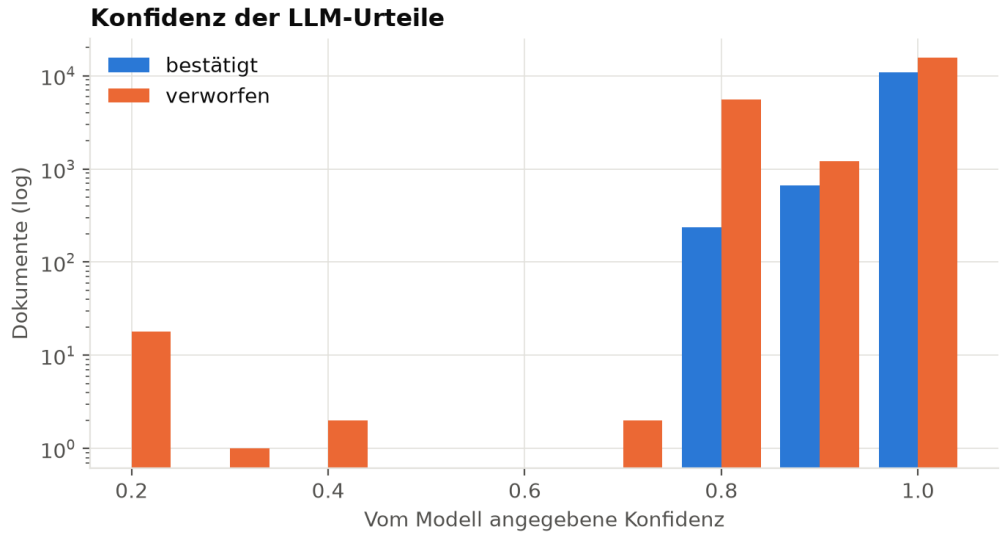


Abbildung D.2: Konfidenzverteilung der LLM-Urteile.

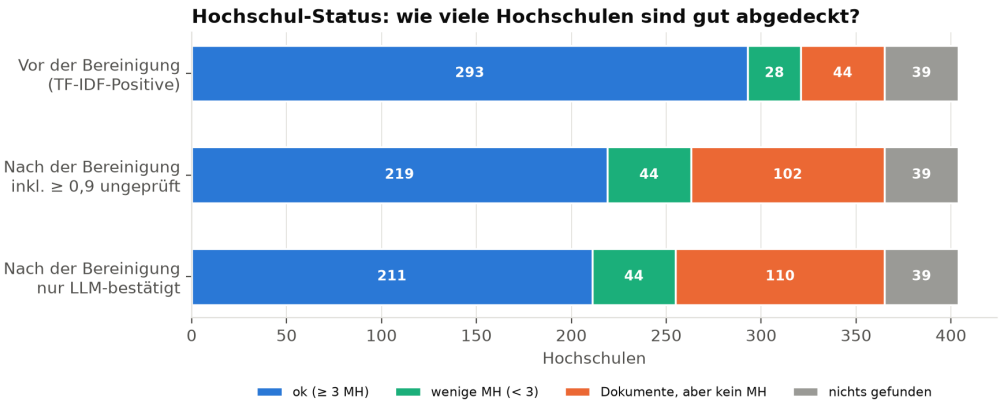


Abbildung D.3: Status der Hochschulen vor und nach der Bereinigung.

E Glossar

Bulk-Crawl
Läufe über viele Hochschulen auf Fargate ohne Zeitlimit.

Discovery
Vorabsuche nach Handbuch-URLs über Suchindizes (Common Crawl, Serper).

Seeds Von der Discovery gefundene URLs, die der Crawler priorisiert holt.

Seeds-only-Modus
Lauf, der ausschließlich Seeds holt und den Visited-Store umgeht.

Visited-Store
Persistenter Speicher (DynamoDB) für bereits gecrawlte Hochschulen.

Review-Band
Score-Bereich zwischen den Schwellen; Dokumente mit unsicherer Entscheidung.

Pilotstichprobe
Zufällige Stichprobe von 400 Positiven mit Score $\geq 0{,}9$ zur Schätzung der Precision der ungeprüften Gruppe.

Strenge / erweiterte Liste
Strenge Liste: nur LLM-bestätigt. Erweiterte Liste: zusätzlich TF-IDF $\geq 0{,}9$ ohne LLM-Prüfung.

SCP Service Control Policy: organisationsweite AWS-Sperre einzelner Aktionen.

KIWIT
Forschungsgruppe zu Funktionen und Folgen Künstlicher Intelligenz in der Wissenschafts- und Hochschulorganisation.

Literaturverzeichnis

[1] Anthropic. Claude haiku 4.5. Modell über Amazon Bedrock, `https://www.anthropic.com`, 2025.

[2] Common Crawl Foundation. Common crawl: Open repository of web crawl data. `https://commoncrawl.org`. Abgerufen im Projektzeitraum 2026.

[3] Martijn Koster, Gary Illyes, Henner Zeller, and Lizzi Sassman. Robots exclusion protocol. RFC 9309, Internet Engineering Task Force, 2022.

[4] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12: 2825–2830, 2011.

[5] Gerard Salton and Christopher Buckley. Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5):513–523, 1988.

[6] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.